

FLIGHT . I

ARCHIVES · MMX

DOSSIER

?

01

EST · MMX

?

DOCUMENTATION

Osionos

Project file

Osionos Team

May 29, 2026

AUTHOR

DATE

REF · ARCH-BE/MINI-BAAS/2026

???

FLIGHT . I

ARCHIVES · MMX

DOSSIER

?

01

EST · MMX

?

DOCUMENTATION

Osionos

Project file

Osionos Team

May 29, 2026

AUTHOR

DATE

REF · ARCH-BE/MINI-BAAS/2026

???

Table of materials

01 Overview

02 Use of AI

2.1 The different usage profiles

03 My philosophy: learn with AI without sacrificing understanding

04 Acknowledgments

05 CHAPTER 1: summary of the skills mobilized

5.1 Typical activity 1: develop the front-end part of a secure web application

5.2 Type activity 2: develop the back-end part of a secure web application

5.3 Transversal skills

06 CHAPTER 2: Presentation of the project

6.1 Presentation of the company and the service

6.2 Project specifications:

6.3 constraints

6.4 Human and technical environment

6.5 Quality objectives

07 CHAPTER 3: Personal projects, front-end

7.1 Application model and diagrams

7.2 Screenshots of user interfaces

7.3 Front-end optimizations and measured results

7.4 Code snippets, static user interfaces (React / SCSS)

7.5 Code snippets, dynamic part

7.6 Accessibility, front security and measurable quality

08 CHAPTER 4. Personal achievements, back-end

8.1 API architecture and data model

8.2 API code snippet, structure and security

8.3 Data Access Component (DAO) Code Snippets

09 CHAPTER 5. Application security elements

9.1 Authentication and role management

9.2 Input validation and injection protection

Osionos ? Project file

.1 API architecture and data model
8.2 API code snippet, structure and security
8.3 Data Access Component (DAO) Code Snippets
09 CHAPTER 5. Application security elements
9.1 Authentication and role management
9.2 Input validation and injection protection
Osionos ? Project file
1 / 101

.1 API architecture and data model

8.2 API code snippet, structure and security

8.3 Data Access Component (DAO) Code Snippets

09 CHAPTER 5. Application security elements

9.1 Authentication and role management

9.2 Input validation and injection protection

Osionos ? Project file

1 / 101

9.3 Front-end and API protections

9.4 Protections against XSS and CSRF

9.5 GDPR Compliance

10 CHAPTER 6. Technological monitoring and security

10.1 Monitoring sources used

10.2 Vulnerabilities identified in the ecosystem

10.3 Potential flaws and corrections to be made

10.4 Conclusion

11 CHAPTER 7. Conclusion

12 Title of resources + references

Osionos ? Project file

2 / 101

9.3 Front-end and API protections
9.4 Protections against XSS and CSRF
9.5 GDPR Compliance
10 CHAPTER 6. Technological monitoring and security
10.1 Monitoring sources used
10.2 Vulnerabilities identified in the ecosystem
10.3 Potential flaws and corrections to be made
10.4 Conclusion
11 CHAPTER 7. Conclusion
12 Title of resources + references
Osionos ? Project file
2 / 101

Osionos was designed in the image of an anthill: organized, structured, and driven by a will collective to achieve a common goal. When watching a time-lapse video from a gallery underground, we see the ants moving quickly, transporting materials, communicating between them. The architecture of these galleries is complex, with interconnected tunnels and chambers that enable smooth movement and efficient storage of resources. An observation is essential: the Ants exploit the resources of their environment to build their habitat. The anthill is a living ecosystem that constantly adapts to its conditions. With colleagues, we went realize that creating an application today requires more and more resources and data ? and that maintaining this ecosystem inevitably involves calling on more resources human or AI. Osionos is a platform that seeks to make this balance visible, in a form accessible and user-friendly.

Linus Torvalds, creator of Linux, needed a version management tool to drive his own project ? that's how Git was born. In the same way, we wanted to create a side project powerful enough to support our future projects.

Overview

The Osionos project was born out of frustration: we were looking for a truly complete dashboarding tool, fast and pleasant to use ? something like Notion, but better suited to our needs. Tool after tool, we encountered the same limitations: lack of customization, insufficient performance, too rigid integrations. We decided to create our own solution. We are fully aware that this is a long-term project.

Use of AI

AI is today an essential element in modern projects, and Osionos does not exception. We integrated it strategically to improve the developer experience and enrich our learning: code generation, debugging, understanding complex problems, standardized documentation, and critical analysis of the progress of the project.

The different usage profiles

Before going into detail, here are the different AI usage profiles that we have identified and experienced.

NOTE TO READER ? BEFORE YOU BEGIN

Two names appear throughout this file, and it is important not to confuse them: - Osionos ? it is the name of the application, the main platform (Notion-like equivalent, where we work with pages, blocks, databases, dashboards). - Prismatic ? this is the name of the public web page: the marketing site / showcase that presents Osionos to the outside world. It's the envelope, not the engine.

In summary: if we talk about the app, it's Osionos; if we talk about the site, it's Prismatic.

For visuals: the screenshots and diagrams integrated into this PDF are in reduced resolution for weight issues. High definition versions as well as source images are available in the GitHub repository: [Univers42/ft_transcendence](https://github.com/Univers42/ft_transcendence) under [wiki/assets/](#). Clickable links in the PDF refer directly to these files.

Osionos ? Project file

source images are available in the

GitHub repository: [Univers42/ft_transcendence](#) under [wiki/assets/](#). Clickable links in the PDF refer directly to these files.

Osionos ? Project file

3 / 101

source images are available in the

GitHub repository: [Univers42/ft_transcendence](https://github.com/Univers42/ft_transcendence) under [wiki/assets/](#). Clickable links in the PDF refer directly to these files.

Osionos ? Project file

3 / 101

PROMPT ENGINEERING

We also tested prompt engineering to generate better quality code. We followed the community's recommendations: few-shot learning, chains of reasoning, precise structuring of instructions. Result: it is a powerful tool, but it takes time to be controlled. The quality of the output is directly correlated to the quality of the prompt.

Some resources that were useful to us:

<https://www.ibm.com/fr-fr/think/prompt-engineering>

<https://www.ibm.com/fr-fr/think/topics/prompt-optimization>

<https://www.promptingguide.ai/en>

AI ASSISTED / COPILOT STYLE

GitHub Copilot is the emblematic example of programming assistance: timely suggestions real, integration into the editor, useful for repetitive tasks. But it does not replace understanding code ? each suggestion must be read and validated.

We explored different models depending on the context. The Artificial Analysis site publishes benchmarks daily newspapers on the main models ? a useful reference for choosing the right tool according to the needs of the

moment. In particular, we noted significant differences between the models in terms of speed, cost and quality of product code. There is no perfect model ? it's context by context.

DEVELOP AGENTIC

The developer agent ? or AI orchestrator ? is a profile that goes beyond traditional assistants. He designs and supervises pipelines of autonomous AI agents: one AI generates code, another verifies the code quality, a third deploys it if the tests pass. We no longer think in individual completions, but in workflows.

It's a demanding profile, but we had to approach it when the team was reduced to two people.

We put in place specialized agents ? one to test, one to resolve the problems identified ? under constant supervision.

Andrej Karpathy describes this paradigm shift well: ?we no longer write code directly 99% of the time. We orchestrate AI agents to do the job, and we focus on supervision and the optimization of these pipelines.?

Main limitation: the cost of AI credits is very high, which quickly slowed down our widespread use. scale.

NO-CODE / AI-NATIVE BUILDER

Tools like Lovable, Replit Agent or Figma AI allow you to build applications entirely in natural language, without traditional code. We tested them for several weeks. They are powerful for rapid prototyping, but their limitations are very real:

Not very flexible on complex projects

Generated code often unmaintainable

Very high long-term technical debt

Slow on large projects

Costly in AI credits

?

?

?

?

?

?

?

?

Osionos ? Project file

?

?

?

?

?

?

?

Osionos ? Project file

5 / 101

?
?
?
?
?
?
?

Osionos ? Project file

5 / 101

REVIEWER / HUMAN-IN-THE-LOOP

This profile treats each AI suggestion as an unverified pull request: nothing is integrated without human audit. It is a careful and effective approach to maintaining code quality while taking advantage of the assistance of AI.

Experiments in communities like GitHub have shown the limits of automation complete: the suggestions could be off-topic, too generic, or saturate the PR ? generating friction and technical debt rather than gain.

My philosophy: learn with AI without sacrificing understanding

This project was born in a particular context. I am part of a generation that did not have the choice to confront AI ? it has established itself in my practices, in my tools, in the job market.

To ignore him would have been to be deliberately late.

But I wanted to be honest with myself on one essential point: using AI does not mean understand less. This project is, I hope, the demonstration of this.

We have voluntarily tested all the usage profiles described above ? not to find the solution. ease, but to concretely understand what each one brings and what it costs. We touched to the limits of vibe coding, measured the gains of prompt engineering, experimented with orchestration of agents. Each time, we read what the AI produced, questioned its choices, corrected its errors, and learned from its approximations.

AI has often forced me to go further in my understanding than a simple course would have done. Understanding why an output is bad means understanding what the good one should have been. I want to be honest about this because I know that the use of AI in school projects is a topic sensitive. My intention was never to bypass learning ? it was to approach it differently, in an era where these tools are already part of everyday professional life. For me, this project is as much a learning of code as a learning of the posture to adopt in the face of AI: curiosity, spirit criticism, responsibility.

Acknowledgments

This project would not have existed without the people who supported it, in difficult times as well as in the good ones.

A big thank you to my team for having held out over the long term, for agreeing to test approaches uncertain, and for continuing to learn even when the road was long. Everyone brought something essential ? an idea, a solution, a presence in moments when we doubted.

Thanks to school 42, which taught us that autonomy and resourcefulness are separate skills whole. This project reflects that.

Thanks to the open source communities, documentation authors, developers who share their online feedback ? you are an invisible but indispensable resource.

And finally, thank you to those who will read this document with open eyes and the curiosity to understand what we have.

really tried to do here.

Osionos ? Project file

le resource.

And finally, thank you to those who will read this document with open eyes and the curiosity to understand what we have.

really tried to do here.

Osionos ? Project file

6 / 101

le resource.

And finally, thank you to those who will read this document with open eyes and the curiosity to understand what we have really tried to do here.

Osionos ? Project file

6 / 101

sorting, and advanced viewing options. The idea here is that we want to be able over time offer people pre-established shapes but if they want they can add their own through the code or even through plugins

home: this is the home dashboard, fully customizable, where users can pin their favorite pages and databases for quick access. The idea is that this welcome dashboard can be shared between members of the same workspace, to create a sort of point of contact common rallying. We will also draw inspiration from what Obsidian did with its ?graph view? to offer a visualization of all the pages and their interconnections. To make the nodes and the edges will be based on the d3.js library, which is very powerful for this type of visualization.

As in Linux everything is an archive? In our system everything is data. Every data can take distinct forms (page, block, database, etc.) and be linked to other data.

The idea is that the graph view can visually represent these connections, to help users navigate their workspace and discover relationships between their data.

databases: users should be able to connect their databases (PostgreSQL, MongoDB, etc.) to Osionos to view and interact with their data in real time. The idea is that users can create personalized views of their data, with filters, sorts, and advanced viewing options. We also want to allow users to create dashboards from this data, to monitor key performance indicators (KPIs) and take informed decisions.

note: of course this system is more or less easy to reproduce what notion does. The difficulty lies more in the infrastructure than in the functionality itself.

wiki: here the observation is that notion is much too slow. Cannot load long page. Obsidian is faster but is only designed for making notes. The general appearance of Obsidian remains very austere, very good tool for professional use but too specific. So we had the idea of manage in infrastructure a way that we can have a static database and a database dynamic. Vscod is surely one of the fastest and most versatile graphics tools but not suitable for everyone. The advantage I see is that you can absolutely do everything with the keyboard and reduce the friction associated with using the mouse. So the wiki will be based on all this friction. The goal is that it is much faster than Notion, more pleasant to use than Obsidian, and more accessible as Vscod. While using real data and letting users choose to write in raw or directly convert the values into our own block or inline markdown.

Objectives

The Osionos project pursued three major and distinct objectives, each linked to a user profile precise and to a problem measured in existing tools (Notion, Obsidian, VS Code, Confluence).

For the end user ? unify the note, the database and the dashboard in a space fast and keyboard-controlled work. The same page had to be able to contain free text, a view tabular connected to a real database, a link graph, and a KPI dashboard, without changing tools nor wait for a page of a thousand blocks to load. The encrypted target: open any page in less than a second, no matter its size.

For the workspace administrator ? have fine-grained, auditable control over the shared space.

The Workspace Planner had to be able to define who sees what (public / private / shared), manage the roles, plug in your own external databases (PostgreSQL, MongoDB, later MySQL and HTTP), and find any critical operation in an audit log. No open secrets, no action without trace.

?

?

?

?

1.

2.

Osionos ? Project file

For the project team ? consolidate general reliability by centralizing authentication, permissions, auditing and observability of the entire platform. Rather than re-writing ten layers of security, we relied on proven bricks (GoTrue for JWT, PostgREST for RLS, Vault for secrets, Kong for ingress) assembled and hardened. The application services that we build runs with a non-root user, and the public flow goes through a single gateway. Each technical choice described in the following section has been validated not only for its ability to deliver these three objectives, but also for its resilience (ability to survive the breakdown of a neighbor) and its security by construction (no application verification when the database can do it itself).

Solution architecture and technical choices

Before explaining what we built, we must say where we started from. At the start of the project, we had a blank sheet and an intuition: ?we want to make a Notion, but who knows how to speak to any base of data, without breaking anything when you change your mind about the infrastructure.? It is this intuition that drove

every technical decision. At each intersection, we chose the option that kept two doors open: that of rapid experimentation for the team, and that of a possible serious production launch.

The rest of this section describes, dimension by dimension, what we chose and especially why we got here ? including the paths we abandoned along the way.

BACK-END SIDE

a. Microservices, all in Docker

Our first reflex was the most classic: a Node/Express monolith with a PostgreSQL database.

This is what we knew, this is what we see in 90% of the tutorials. We lasted two weeks. The problem appeared very quickly: as soon as we wanted to add MongoDB for the flexible Osionos blocks, then Redis for cache, then MinIO for files, the monolith started to look like a bag of nodes where each dependence tugged on the others. A bug in the file layer caused auth to fail. A restarting to add an environment variable shut down the entire app.

We went back and established a simple rule: each responsibility has its container, and the Application services that we build have a Dockerfile reproducible with a non-root user.

From there, the stack began to take shape naturally ? a brick for auth (GoTrue), a for the relational database (PostgreSQL), one for the documents (MongoDB), one for the cache (Redis), one for files (MinIO), one for the gateway (Kong), and a series of NestJS microservices for logic that belongs to us (mongo-api, query-router, storage-router, permission-engine, gdpr-service, etc.). Each service is isolated and restartable independently. Images are versioned as much as possible; an identified floating tag (realtime-agnostic:latest) remains a release debt to be corrected before real production.

The trigger was to understand that not rewriting what already exists is in itself a skill. We was not going to redo a PostgREST or a GoTrue which are better than what we could have produced in two month. We assembled them and hardened them.

b. Data federation

The real ambition of Osionos is to let a user connect their database, whether it is PostgreSQL, MongoDB, MySQL or whatever, and navigate through it as if it were a Notion page.

At the beginning we tried the naive approach: one connector per engine, hard-coded in the front. It worked for one, painful for two, unbearable for three.

3.

Osionos ? Project file

, hard-coded in the front. It worked
for one, painful for two, unbearable for three.

3.

Osionos ? Project file

11 / 101

, hard-coded in the front. It worked
for one, painful for two, unbearable for three.
3.

Osionos ? Project file

11 / 101

We realized that we were reinventing a known problem: that's exactly what resolve SQL federation engines. We evaluated Presto, Trino, Apache Drill, and even a few proprietary options. We chose Trino for two reasons: it is open source, and it can read PostgreSQL and MongoDB with the same SQL syntax, paving the way for queries that join the two worlds ? something that would have required weeks of glue code for us.

But Trino is an analytical engine, not transactional. For classic business operations (create a block, modify a page), we needed a short and secure path. We therefore built a in-house service, the query-router, which takes an abstract query description (list , insert , update , etc.) and translates it to the correct engine via specialized adapters. The beauty of the approach is that adding a new engine only requires a new adapter ? no redesign of the rest.

c. Multi-engine consistency

Very quickly, an annoying question arose: if the user writes in PostgreSQL and we want to pass on this writing in MongoDB (for example to update a denormalized view), how do we ensures the two stay in sync? The intuitive answer ? ?we do both writings in the same transaction? ? is physically impossible as soon as we cross two different engines. It's a theorem, not a lack of effort.

We looked at how the big platforms solve this problem. Supabase, Hasura, AWS all apply the same recipe: the outbox pattern. Application writing is done in a single database (the ?source of truth?), plus a row in an event table. A relay service reads these events and replays them to other systems ? Mongo, Elasticsearch, external webhook, etc. We loses immediate atomicity, but gains eventual guaranteed consistency, plus audit and replay free.

This is what we retained for the M3 milestone of the roadmap, relying on Redis (already present) as future event bus via Redis Streams ? no need to add Kafka until project scale not impose it. Today, Redis is mainly used for the query-router application cache; the outbox and the relay remain to be delivered.

d. Unified API and client SDK

A platform that exposes ten different services with ten different conventions is unmanageable. We wanted the front-end developer (ourselves, in this case) to have only one way to talk to the back, no matter what happens behind the scenes.

We looked at the APIs we liked to use: Supabase, Firebase, PocketBase. The common point is a SDK that looks like `client.from('table').select().eq(...)` ? close to SQL, but portable, typed, and independent of the engine. We took this idea and wired it to our query-router. The result, this is our SDK @mini-baas/js, which is consumed by the two Osionos frontends without them having to care about whether the data comes from PostgreSQL, Mongo or an external database saved by the user.

On the gateway side, we chose Kong in YAML declarative mode, without a database. It is more rigid than a Classic Kong, but that means that the entire ingress configuration lives in Git, therefore reviewable and reproducible. A new route is not deployed by a click in a UI, it goes through a pull request ? this is exactly the guarantee we were looking for.

e. Built-in security and observability

Osionos ? Project file

s through a pull request

? this is exactly the guarantee we were looking for.

e. Built-in security and observability

Osionos ? Project file

12 / 101

s through a pull request

? this is exactly the guarantee we were looking for.

e. Built-in security and observability

Osionos ? Project file

12 / 101

Two guiding principles emerged. On security, the least privilege is placed at the lowest level possible: it does not live in ifs scattered in the code, it lives in the base via RLS policies from PostgreSQL. Even if the entire application layer was bypassed, PostgreSQL would refuse the read. Regarding secrets, we very quickly left the versioned .env in favor of HashiCorp Vault as soon as we started handling external database credentials ? a .env in git was a risk unacceptable.

On observability, the team spent enough nights debugging blindly to make it a priority. Prometheus + Grafana + Loki + Promtail are in place today; distributed traces (OpenTelemetry + Tempo) are planned in M4 ? the architecture is already wired to accommodate them. The details of the defense layers and tools are consolidated below in the Defense Strategy section. security.

f. Development and deployment tooling

A stack of 36 Docker services becomes incomprehensible without good tools. We deliberately invested in the tooling from the start, following three principles: everything is in Docker Compose (a new dev does make baas-up and finds the same topology), everything can be tested locally (16 test phases, including 15 shell scripts and a Python phase, validating auth, RLS, isolation, storage, realtime, etc.), everything is reproducible (docker-bake.hcl multi-arch, idempotent migrations, images traced by version when they are published).

We deliberately resisted Kubernetes: as long as the stack fits on a Docker Compose machine, we keeps complexity minimal. The complete list of tools is in the Tools section of development.

g. Auditability and traceability

The last choice which structures the entire back is less glamorous but perhaps the most important: everything What happens in the platform must be able to be explained after the fact. Each HTTP request receives a X-Request-ID at Kong entry, propagated to the base. Each migration is numbered and traced.

The audit_log table (which keeps actor, action, resource, payload) is being generalized to all the writings (milestone M1) ? combined with the Loki logs, it gives a platform where ?what happened to 2:32 p.m. yesterday for user X? » becomes a question of a minute, not a day.

This discipline of the trace is a discipline of respect for the future of the team: we know that we will forget, and we build the memory of the platform while we still have the context in mind.

FRONT-END SIDE

The front was designed with a different logic from the back, because the constraint is not the same: on On the front, the number one enemy isn't data consistency, it's user friction. A page that takes a second too long to respond, it's a lost user. An interface that we don't I don?t understand, it?s an abandoned project.

Two frontends, two philosophies

We found ourselves early on needing two very different things: (1) a quick-to-use marketing site. charge, well referenced, sober, which presents Osionos to the world ? it is opposite-osiris; (2) a Dense, interactive, near real-time product application that looks more like an IDE than a site ? it?s osionos. Wanting to solve both with the same stack would have been a mistake.

Osionos ? Project file

tion that looks more like an IDE than a site ?

it?s osionos. Wanting to solve both with the same stack would have been a mistake.

Osionos ? Project file

13 / 101

tion that looks more like an IDE than a site ?

it?s osionos. Wanting to solve both with the same stack would have been a mistake.

Osionos ? Project file

13 / 101

For marketing, we chose Astro. The reason is simple: Astro produces static HTML by default and only uses JavaScript when it is strictly necessary (the famous "islands architecture"). For a site whose objective is to load quickly and be well indexed by search engines, this is the best choice most rational available today. We accepted not having the React ecosystem for that ? and it was the right decision.

For the product application, we went with React 19 + Vite. React because that's what the team mastery, that the doc is easily found and that the ecosystem (tanstack, simplewebauthn, etc.) is without rival. Quickly because after having suffered on Webpack and Create-React-App in other projects, we had no longer want to wait thirty seconds for each save. Vite starts in a second and recompiles in less than a hundred milliseconds ? this is non-negotiable when developing a complex UI.

Why not Next.js

The question came up three times during the design: "what if we did everything in Next.js, marketing and app, in a single project?". We dug, and we pushed aside. Three reasons:

Next.js forces a certain rendering vision (SSR / RSC) which complicates integration with our BaaS self-hosted. We wanted a client that spoke to our gateway, not a framework that presupposed Vercel. The cost of learning React Server Components seemed disproportionate to us compared to the gain for a team of five becoming two.

Separating marketing (static) and app (SPA) gives us two simpler build pipelines, two clear mental scopes, and the ability to iterate on one without breaking the other.

State management: Zustand rather than Redux

We tried Redux Toolkit at first. It's powerful, but it's also three files to touch to add a field to a blind. On the Osionos scale (dozens of states: current page, block in edit, filters, views, multiple selection, etc.), the friction became unbearable. We migrated to Zustand, which fits into one function per store and which sticks to the mental model of React. We paid for this choice with the obligation to discipline our selectors (React 19 is strict on stable snapshots) ? but it's a compromise that the team agreed.

Architecture in Feature-Sliced Design

When we realized that we were going to exceed a hundred components, we established an explicit architecture: Feature-

Sliced Design. It is a public convention that organizes code into layers (entities , features , widgets, pages, app) with strict rules on who has the right to import who. The benefit was seen immediately: we no longer wonder where to put a new component, the convention answers. And one new team member knows how to read the structure without having to explain it to them.

The SDK as a contract

The front never talks directly to PostgreSQL or Mongo. It talks to our SDK @mini-baas/js, which speaks to Kong, who dispatches to the right department. This is voluntary: it means that changing the back does not

break the front, as long as the SDK contract remains stable. This indirection has a cost (a layer additional to maintain), but it has already saved us twice: once when we shifted from Supabase hosted to our self-hosted BaaS, and once when we redesigned the session format.

Accessibility and perceived performance

- 1.
- 2.
- 3.

Osionos ? Project file

ed the session format.

Accessibility and perceived performance

- 1.
- 2.
- 3.

Osionos ? Project file

14 / 101

ed the session format.

Accessibility and perceived performance

1.

2.

3.

Osionos ? Project file

14 / 101

Two things that we dealt with from the start and not at the end of the project: accessibility and performance perceived. We learned along the way that adding accessibility at the end costs ten times more than adding accessibility.

think up front ? and the same goes for performance. Concrete techniques

(virtualization, code splitting, suspense, ARIA tokens, focus trap) are described in the section

Code performance and quality.

In summary, the Osionos architecture was not designed in one go on a whiteboard. She got constructed by accumulation of decisions, each taken in reaction to a real wall that we encountered.

This is probably what makes it coherent: there is almost no layer that we could not have justified by a concrete problem that we have experienced.

SUMMARY OF STACK CHOICES

Rather than rolling out a catalog, the simplest thing is to tell the stack by large families, because that each family answers a specific question that we asked ourselves at the start.

On the user interface side, we separated the site which presents Osionos and the application we use. The site marketing is in Astro, because it must load quickly and reference well; the application is in React 19 + Fast, because that's what allows us to maintain a dense editor without becoming slow. Between the two, we shares an internal SDK @mini-baas/js: it is he who speaks to the back, and it is he who guarantees that we can change a brick on the server side without breaking the front. The browser-side state goes through Zustand, code organization by Feature-Sliced Design, and passwordless authentication by WebAuthn.

On the core side of BaaS, we assumed not to rewrite what already exists: Kong as the only entry point, GoTrue for authentication, PostgREST to expose PostgreSQL in REST with RLS as a guard final fool, and PostgreSQL as the source of truth. Besides, MongoDB is used for semi-structured blocks, with a mongo-api house facade that automatically injects the owner from the JWT. Redis serves as a cache and future event bus. MinIO stores the files, and our storage-router generates pre-signed URLs so that uploads never cross our services.

Around it, we wrote a handful of NestJS services which carry the logic that belongs to us: query-router to dispatch requests to the right engine, permission-engine to centralize the rules, session-service for session lifecycle, schema-service for introspection multi-engine, gdpr-service for export and anonymization, plus some utility services (logs, email, newsletter, AI, analytics). Trino connects to PostgreSQL and MongoDB for reading enable cross-engine analytical queries without breaking the transactional path.

Finally, security and operation rely on an nginx + ModSecurity WAF upstream of Kong, HashiCorp Vault to retrieve or generate secrets outside Git, AES-256-GCM encryption for external database credentials, and Prometheus, Grafana, Loki, Promtail for observability. Everything is orchestrated in Docker Compose, built via docker-bake.hcl, published on GHCR and Docker Hub with version tags, and covered by a system test suite organized by phases 1 to 16 before merges important.

The tables below are mainly for quick reference; they are not meant to be read in full in one shot.

Front-ends and SDK

Osionos ? Project file

15 / 101

important.

The tables below are mainly for quick reference; they are not meant to be read in full in one shot.

Front-ends and SDK

Osionos ? Project file

15 / 101

important.

The tables below are mainly for quick reference; they are not meant to be read in full in one shot.

Front-ends and SDK

Osionos ? Project file

15 / 101

CONNECTION SCENE: FROM CLICK TO TOKEN SESSION

The diagram below follows a user who opens the opposite-osiris marketing site, clicks ?Se connect?, arrives on osionos, authenticates, and receives the session which opens his workspace. Each arrow is a real interaction; each service occurs at a specific time for a reason precise.

Reading the diagram: the user never speaks directly to a base. The public feed goes through WAF ? Kong, which assigns an X-Request-ID and enforces input controls. GoTrue validates credentials and signs a JWT with a JWT_SECRET provided by the environment, itself generated or retrieved by the Vault/Makefile scripts outside of Git. The refresh token is protected on the application gateway side by a cookie HttpOnly; Secure; SameSite=Lax. The session service exists, but its sessions persist in PostgreSQL (session.user_sessions).

Redis is used for the query-router cache and as the basis for the future event bus. On the business query that follows, the JWT is replayed: PostgREST passes it to PostgreSQL, which applies automatically the RLS (auth.uid() = owner_id) ? the final security is in the database, not in the application code.

OVERVIEW OF CONNECTIONS BETWEEN SERVICES (CURRENT STATUS)

The diagram below reflects the actual state of mini-baas-infra's docker-compose.yml at the time of writing. The services are grouped by execution plan (Compose profiles): control-plane, data-plane, adapter-plane, background, storage, analytics, observability.

Brick

Role

GHCR + Docker Hub

Distribution of images with version tags; pinning by digest to be completed before strict production

Phased BaaS Suite

System testing: phases 1 to 16, with 15 shell scripts and a Python phase

Osionos ? Project file

17 / 101

profiles): control-plane,
data-plane, adapter-plane, background, storage, analytics, observability.

Brick

Role

GHCR + Docker Hub

Distribution of images with version tags; pinning by digest to be completed
before strict production

Phased BaaS Suite

System testing: phases 1 to 16, with 15 shell scripts and a Python phase

Osionos ? Project file

17 / 101

profiles): control-plane,
data-plane, adapter-plane, background, storage, analytics, observability.
Brick
Role
GHCR + Docker Hub
Distribution of images with version tags; pinning by digest to be completed
before strict production
Phased BaaS Suite
System testing: phases 1 to 16, with 15 shell scripts and a Python phase
Osionos ? Project file
17 / 101

How to read this diagram:

Client side ? two independent frontends share the @mini-baas/js SDK. No direct call to the data from the browser.

Network perimeter ? any request goes through waf (OWASP CRS filtering) then kong (routing, JWT, rate-limit, CORS). Only public entry point.

control-plane ? governance: gotrue, vault, session-service, permission-engine, schema-service, pg-meta, adapter-registry, studio.

data-plane ? engines and their facades: postgres behind postgres, mongo behind mongo-api, realtime which listens to WAL + change streams, supervisor which pools PG.

adapter-plane ? query-router consults adapter-registry to dispatch CRUD to the correct one engine.

storage ? storage-router talks to minio with environment-injected S3 credentials;

the encryption of external database credentials is carried by adapter-registry.

background ? long-life services: email-service, newsletter-service, gdpr-service, ai-service, analytics-service, log-service.

analytics ? trino with PG + Mongo catalogs, for cross-engine analytical queries.

observability ? prometheus, grafana, loki, promtail. Distributed traces (Tempo/OTel) are not yet in place, see target 10/10 below.

The traffic rule remains the same: the arrows always go down from the least privileged to the most privileged. The client only knows Kong, Kong only knows the application services, the services applications only know their engine. A compromise on one level does not propagate to the next without crossing a new barrier (JWT, RLS, ACL MinIO, secret injected from Vault or variable dedicated environment).

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.

Osionos ? Project file

ations only know their engine. A compromise on one level does not propagate to the next without crossing a new barrier (JWT, RLS, ACL MinIO, secret injected from Vault or variable dedicated environment).

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.

Osionos ? Project file

18 / 101

ations only know their engine. A compromise on one level does not propagate to the next without crossing a new barrier (JWT, RLS, ACL MinIO, secret injected from Vault or variable dedicated environment).

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.

Osionos ? Project file

18 / 101

TARGET 10/10 ? WHAT THE INFRASTRUCTURE WILL LOOK LIKE ONCE THE M1-M5 MILESTONES ARE DELIVERED

The diagram below represents the target state after the five milestones described in [wiki/todo/README.md](#) carried out. New developments compared to the current state are grouped in subgraphs M1 to M5.

Everything that appears outside of these blocks already exists today.

What milestones actually add:

Milestone

Contribution to the diagram

Why is it necessary to pass

at 10/10

M1 · hardening

HEALTHCHECK on all services, interface

IDatabaseAdapter, OpenAPI 3.1 spec

versioned, audit_log PG table

Make the stack self-described

(Compose no longer tolerates service mute) and trace each writing

M2 · federation

extent

mysql-engine, redis-engine, http-

engine + corresponding Trino catalogs, register

of encrypted external DBs

Deliver on the ?connect? promise

any basis?, not

only PG + Mongo

M3 · consistency

multi-engine

Outbox table, debezium connect, Redis

Streams like bus, outbox-relay,

Idempotency-Key middleware

Guarantee possible consistency

between engines without rolling

distributed transaction

M4 ·

observability

complete

OpenTelemetry collector, Tempo for traces

distributed, Alertmanager + runbooks

Being able to follow a stub query

at the end (Tempo absent today)

and be alerted before the user

M5 · security

hardened

Kong plugins OPA + OIDC, helmet + strict CSP

front side, automatic JWT rotation, SAST/DAST

in CI (Semgrep + ZAP)

Switch from default security

acceptable to security by

audited construction

Osionos ? Project file

19 / 101

uction

Osionos ? Project file

19 / 101

The bricks already present (gateway, auth, RLS, Vault, partial observability, PG/Mongo federation, Trino, GDPR, light application audit) are not replaced: they are supplemented and hardened. None milestone requires no rewriting, only targeted additions ? that's what makes the path to 10/10 realistic at constant numbers.

Development tooling

With a reduced staff ? five at the start, two at the end ? we did not have the luxury of juggling ten channels different tools. We therefore made everything pass through the same base, by imposing a simple rule: if a command does not execute the same on my machine, on that of a teammate and in the CI, it is because it is not finished.

The common base is Docker Compose. The entire stack ? front, BaaS, observability, tools ? starts from the same docker-compose.yml , with multi-architecture builds orchestrated by docker-bake.hcl and published on GHCR and Docker Hub with version tags. Strict pinning by digest remains a hardening target: the stack still contains a floating realtime-agnostic:latest tag, identified as debt before production. On top of that, a Makefile serves as a single facade: make baas-up , make baas-test , make osionos-dev , make certs-doctor . A new member does not need knowing each service to be productive, it needs to know the make targets.

The application tools are deliberately homogeneous in TypeScript. The product front uses React 19 + Quickly 6, because we wanted almost instantaneous HMR and reliable end-to-end testing with Playwright. The marketing site uses Astro 6, because it must load quickly and be referenced well. Business micro-services are in NestJS, because the module/controller/service format gave a clear framework without imposing a architecture too heavy. Dependencies are managed in pnpm with workspaces, which saves us from having to recompile the same thing ten times in CI.

Static quality goes through ESLint and SonarQube/SonarCloud depending on the packets, with Prettier configured

at least on the BaaS NestJS workspace. Dependabot is configured weekly and Renovate maintains a dashboard of grouped and deferred updates. Dynamic quality requires a suite of system tests organized by phases 1 to 16 (under apps/baas/mini-baas-infra/scripts/) which validates end to end

end authentication, RLS, user isolation, JWT lifecycle, storage, realtime, rate-limit and CORS. The project rule is to run make baas-test before major merges; the CI BaaS now replays a critical subset of the phases.

Security strategy

Osionos' security was not added at the end like a veneer; it is laid in layers successive, with a constant rule: if one gives way, the next must still hold. This is what we call defense in depth, and concretely that gives four levels.

The first level is on the outskirts. An nginx WAF equipped with ModSecurity and OWASP CRS rules examines each request before it even reaches our services: known SQL injections, XSS, Aggressive scanners and protocol anomalies are filtered upstream. Just behind, Kong carries the rate-limit, CORS, validation of JWTs and the propagation of an X-Request-ID which can be found even in the database logs.

The second level concerns identity. It is GoTrue which issues the JWTs, with a signature key provided by the runtime environment and retrieved or generated by the Vault/Makefile workflow ? never by a .env versioned. The passwords are hashed on the GoTrue side, and we also wired the WebAuthn Osionos ? Project file

v versioned. The passwords are hashed on the GoTrue side, and we also wired the WebAuthn

Osionos ? Project file

20 / 101

v versioned. The passwords are hashed on the GoTrue side, and we also wired the WebAuthn

Osionos ? Project file

20 / 101

(passkeys) on the marketing site to provide a password-free path. When a user connects their own external database, its credentials are encrypted at rest in AES-256-GCM with derivation script, because we consider that a local leak should never be enough to compromise third-party access. The third level lives in the data itself. On the PostgreSQL side, it is the RLS policies which have the last word: as long as `auth.uid() = owner_id` is not satisfied, the database refuses to serve a line, even if the entire application layer was bypassed. On the MongoDB side, it is the mongo-api service which automatically injects `owner_id` on every write from the JWT. And all SQL queries pass either by PostgREST or by parameterized queries, which makes SQL injection structurally impossible rather than simply ?not observed?.

The last landing is on the front side. User inputs are not rendered raw: the marketing site uses `sanitize-html` where it accepts HTML, and the Osionos application escapes HTML in `markengine` with URL filtering (`sanitizeUrl`). The access token lives in memory for calls Authorization: Bearer , while the refresh token is stored in an `HttpOnly` cookie; Secure; SameSite=Lax on auth-gateway side. Accessibility (RGAA) is addressed from the design ? HTML semantics, contrast, visible focus, full keyboard navigation ? and GDPR compliance is supported by the GDPR-service, which actually exposes the export, anonymization and deletion endpoints, rather than to be a simple sticker in the footer.

Everything is completed, without noise, by the obligatory code reviews on GitHub, the continuous scanning of dependencies via Renovate, and the isolation/auth portion of the system test suite which replays regularly the most common attack scenarios.

Code performance and quality

Osionos performance is not an abstract subject: a real workspace can contain thousands of blocks, and the page must remain fluid even in this case. The rule we gave ourselves is that no page must not row because it has become serious.

The first lever is virtualization. Long lists ? blocks of a page, rows of a database view data ? passes through `@tanstack/react-virtual` , which only renders in the DOM what is actually visible. We can thus scroll through thousands of elements without loss of fluidity. On the server side, PostgREST carries pagination via Range headers, which avoids downloading everything to display than a window.

The second lever is cache and latency. Redis is used for the query-router cache and prepares for the future event bus; application sessions that pass through session-service are persisted in PostgreSQL. File uploads never pass through our application services: MinIO generates URLs pre-signed and the client uploads directly, which removes a potential bottleneck.

The third lever is deferred loading. Quickly cut the bundle by route, React 19 and Suspense report non-critical sections, and the Astro marketing site loads zero JavaScript by default. A Visitor who arrives on a product page does not have to pay the cost of the entire application before being able to read.

On the code quality side, we relied on explicit conventions rather than discipline individual. The front follows Feature-Sliced Design with strict import rules between layers, BaaS is divided into NestJS micro-services by domain, and everything goes through the SDK `@mini-baas/js` which serves as

stable contract between the two worlds. The documentation remains alive ? this wiki, the README.md by service, Mermaid diagrams ? and comments are focused where the ?why? is not readable in the code: RLS, encryption, query-router dispatch. The rest is meant to read on its own.

Osionos ? Project file

, encryption, query-router dispatch. The rest is meant to read on its own.

Osionos ? Project file

21 / 101

, encryption, query-router dispatch. The rest is meant to read on its own.

Osionos ? Project file

21 / 101

LONG-TERM VISION

In the long term, Osionos could become a sort of universal data workstation for a company.

team: a place where we write, where we connect bases, where we visualize, where we automate, and where you can ask for help from an AI without leaving your work context.

The vision is not just to remake Notion. Notion manages the page very well. Obsidian handles the local note. Retool manages the business interface very well. Supabase manages the application backend very well.

Osionos? ambition is to seek the intersection: a human-readable work interface, connected to real data, with an infrastructure that the team can understand and own.

The main risk of this development is obvious: wanting to do everything and ending up finishing nothing. It is for this means that the roadmap must remain strict: each new capacity must either improve the real use of a team, or strengthen the reliability of the platform. If she doesn't do either, she has to wait.

the constraints

The development of Osionos was framed by strong constraints, both academic, technical, safety and quality. This was not a project that you could just launch with npm install on a personal machine and correct by feeling. The work environment is directly inspired by the spirit of the Born2beroot / Inception projects of 42: a strict virtual machine, a network exposure limited, isolated services, and one simple rule ? everything that runs must be repeatable.

Reproducibility was not a ?nice to have?: it was the condition for the project to survive the transition from one machine to another, from one OS to another, and on the day of the evaluation. This is why we pushed the idea

until the end: the reference VM itself is versioned in a dedicated repo, Univers42/

born2root . This repo allows you to regenerate, from scratch, a modern and hardened VM (Debian + Docker + firewall + users + SSH) which then serves as a basis for cloning and launching ft_transcendence / Osionos. It?s a real inception: a reproducible VM that hosts a reproducible Docker stack.

A. ENVIRONMENTAL CONSTRAINTS: STRICT VM, DOCKER EVERYWHERE, ZERO LOCAL DEPENDENCE

The most structuring constraint was the execution environment. The repo explicitly documents that the stack must go through Docker Compose only: you must not install the dependencies applications on the host, nor start the website or Osionos with local npm, pnpm or node scripts. The README.md file indicates that the root docker-compose.yml is the source of truth for the backend, the marketing site, the Osionos application and the bridges.

In practice, the development was done in a b2b VM under VirtualBox ? the VM built from from the Univers42/born2root repo ? with Docker inside the VM and sometimes the browser on the host machine. This created a real network constraint: the full path became the host browser - > localhost host -> NAT VirtualBox -> VM -> Docker ports -> HTTPS proxy -> container .

The document wiki/host-browser-https-pipeline.md explains this pipeline in detail. A green stack in Docker was not enough: the ports also had to be published on 0.0.0.0, the local certificate is recognized by the browser, and that the user does not open a VS Code port randomly transferred to the port square Compose canonical.

This constraint forced us to automate many things: generation of local certificates, import of CA into system and browser stores, check ports, make all as pipeline main, and container-only.mjs on the opposite-osiris side to prevent execution outside the container.

Osionos ? Project file

ntainer-only.mjs on the opposite-osiris side to prevent execution outside the container.

Osionos ? Project file

26 / 101

ntainer-only.mjs on the opposite-osiris side to prevent execution outside the container.

Osionos ? Project file

26 / 101

B. SECURITY CONSTRAINTS: SENSITIVE DATA, SECRETS, GDPR

Even if Osionos is not a field planning application like the GeoTask example, it manipulates still sensitive data: user accounts, sessions, private workspaces, roles, databases external data connected by the user, connection strings, files, logs and activity traces. The constraint was therefore not only to protect a login form, but to protect a data ecosystem.

Concretely, several obligations were imposed. Authentication had to be strong: GoTrue supports registration, login, password hashing and JWT issuance, with clear separation between the anon, authenticated and service_role roles. Data isolation does not could not rely on the goodwill of the application code: it is PostgreSQL which decides, via the RLS (`auth.uid() = owner_id`), and it is MongoDB which systematically receives an `owner_id` injected by mongo-api from the JWT. Secrets never have a place in Git: they are generated or retrieved via HashiCorp Vault, then injected into services by the runtime environment; basic credentials External devices that users plug in are encrypted at rest in AES-256-GCM with scrypt bypass. Finally, ingress protection combines WAF nginx, ModSecurity and OWASP CRS upstream of Kong, with strict CORS, rate-limit and header control, while the front applies sanitize-html on the relevant surfaces of the marketing site and HTML/URL escaping in the Markdown engine of the app. The GDPR is not treated as a post-project administrative task: the GDPR-service exposes actually the export, anonymization and deletion endpoints, with minimization logic and traceability.

A specific constraint concerned the shared recovery of secrets. At the beginning, each machine generates its own local `.env`, and that's enough to work on its own. But as soon as we wanted to start the stack on a teammate's machine or in a fresh VM, we realized the problem: we didn't have the right to send real JWT keys, OAuth credentials or SMTP secrets by message, and we do not couldn't version them either. There needed to be a way to share the same sensitive values without never expose them clearly.

The solution is based on two uses of HashiCorp Vault. Locally, Vault runs in the docker-compose.yml root via the secrets profile and remains accessible behind the local HTTPS proxy `https://localhost:18200`; the scripts and some bootstrap containers talk to it on the Docker network internal (`http://vault:8200`). For team sharing, we deployed a shared instance on Fly.io, exposed to the HTTPS address `https://track-binocle-vault.fly.dev` via `make vault-fly`. This instance is not used to host the application ? it only serves as an access point to secrets shared.

The typical path looks like this: a maintainer generates a token with `make vault-fly-invite-token VAULT_TEAM_ROLE=reader`, optionally chooses a lifetime short, and transmits this token via a secure single-use channel (typically OneTimeSecret). The developer places the ignored file `.vault/track-binocle-reader.env` in his clone, passes it `chmod 600`, runs `make vault-shared-doctor` to check the wiring without showing any values, then simply `make all`: the Makefile contacts Vault over HTTPS, retrieves the variables authorized by the policy associated with the token, and generates the local `.env` in the correct subfolders. If anyone tries to share a localhost token, the Makefile refuses (unless explicitly waived for testing on the same machine), because such a token does not prove anything on another VM. On the CI side, GitHub Actions does not store never a static Vault token: the pipeline is authenticated by OIDC and receives a temporary token of no value than the time of a run.

Osionos ? Project file

receives a temporary token of no value
than the time of a run.

Osionos ? Project file

27 / 101

receives a temporary token of no value
than the time of a run.

Osionos ? Project file

27 / 101

The overall philosophy remains the same: a secret can be recovered, but only with a token valid, private, limited by a policy, possibly expiring, and never versioned. It is in this logic where the broader rule fits: never rely on security on a single layer. If the forehead wrong, the gateway must still filter. If an application service makes a mistake, PostgreSQL must still refuse. If a local file leaks, it must not contain a plaintext secret.

This diagram shows the logic of defense in depth: no layer is considered as sufficient alone. The front reduces risk, the WAF filters, Kong controls the entry, GoTrue carries the identity, the Services apply business rules, and the database has the final say on actual access to data.

C. ARCHITECTURAL CONSTRAINTS: NOT A CLASSIC EXPRESS API

Another structuring constraint was not to reduce Osionos to a React / Node.js / stack.

PostgreSQL. The project uses React and PostgreSQL well, but behind the front there is an assembly: Kong, GoTrue, PostgREST, PostgreSQL, MongoDB, Redis, MinIO, Vault, Trino, plus several NestJS microservices which carry the business logic. This assembly imposes a fairly strict discipline.

The first rule is that all public traffic goes through WAF then Kong. No front speaks directly to a base; each data engine has its controlled facade (PostgREST, mongo-api, storage-router), and it is this facade that carries authentication and isolation. The second rule, is that each service must be able to live separately: isolated in its container, configurable by environment variables, and restartable without interrupting the rest of the platform. The third rule is that the gateway configuration must be readable in Git: Kong runs in DB-less mode with its configuration in versioned YAML, so any modification of routes or plugins goes through a review of code, not by a clickable UI. Finally, the stack must be able to start locally without depending on an external cloud: a developer on his VM must have exactly the same platform as in CI.

This constraint made the project more complex ? it would have been quicker to paste everything in an Express monolithic ? but that's also what makes it coherent. We didn't just build an application, we has built a small platform, and each service can be justified by a concrete problem that we have met.

D. FRONT-END QUALITY CONSTRAINTS

On the front side, the constraint was twofold: produce a truly rich interface, but without sacrificing maintainability. Osionos is a dense tool, with pages, blocks, drag and drop, menus contextual, dashboards and lots of keyboard interactions. Even the smallest broken UX detail can make the tool is painful to use, and it quickly becomes impossible to repair if we have not put in place safeguards as soon as possible.

Our safeguard is therefore a chain of controls which runs before each merge, and which passes entirely in Docker via apps/osionos/app/scripts/docker-run.sh. TypeScript blocks errors type with tsc --noEmit . ESLint is configured with --max-warnings=0 , so we do not tolerate any warning. Playwright plays end-to-end user scenarios, canvas tests verify the Osionos ? Project file

type with tsc --noEmit . ESLint is configured with --max-warnings=0 , so we do not tolerate any warning. Playwright plays end-to-end user scenarios, canvas tests verify the

Osionos ? Project file

28 / 101

ype with tsc --noEmit . ESLint is configured with --max-warnings=0 , so we do not tolerate any warning. Playwright plays end-to-end user scenarios, canvas tests verify the Osionos ? Project file

28 / 101

block behaviors and markdown analysis, bridge tests validate the connection between Osionos and BaaS, and a set of UX/browser tests covers focus management, drag and drop, inline toolbar, menus, indentation, paste, assets and context menu. Finally, a doctor verifies that the test environment is correct before trusting the results ? because a test that passes in a broken environment doesn't prove anything.

The rule that emerges from all this is constant: the front quality does not depend on the machine of the developer. If the pipeline does not work the same for me, for a teammate and in CI, we consider that the pipeline is not finalized.

E. QUALITY CONSTRAINTS ON THE BACK-END AND INFRASTRUCTURE SIDE

On the BaaS side, we couldn't just do classic unit tests, because most of the Risk does not live in an isolated function ? it lives in integration between services. When Kong, GoTrue, PostgREST, PostgreSQL, MongoDB, Redis, Vault, MinIO and realtime must work together to whether a user simply reads their own page, the risk is in the seams, not in the bricks.

We therefore set up a dedicated local CI, in `apps/baas/mini-baas-infra/scripts/run-ci-local.sh`, which first checks the prerequisites (Docker, Docker Compose, Make, curl), validates the Bash syntax of all scripts and passes ShellCheck when available. It then completely cleans up the Compose state to do not inherit from an old volume, generate a deterministic `.env`, start the stack, play the db-bootstrap, checks the health of the gateway on `/auth/v1/health`, then runs make tests.

This make tests of the mini-BaaS chains the `phase*-.sh` / `phase*-.py` scripts in order, and each phase covers a specific risk: smoke tests, authentication, authenticated DB access, isolation user, HTTP methods, error codes, token lifecycle, storage, complex mutations, realtime WebSocket, rate-limit, CORS, Mongo MVP, full auth flow. Next, SonarCloud is configured via `sonar-project.properties`, and vendor/QA plays the role of test registry: it catalogs the scripts existing data and stores their results.

The back quality constraint therefore did not boil down to ?the roads respond?. She was more demanding: the platform must be able to be destroyed, rebuilt, tested and explained, without manual intervention fragile between stages.

F. METHOD AND PLANNING CONSTRAINTS

The project started with a team of five people, then gradually expanded. This has imposed strong prioritization: everything could not be finished at the same time. We therefore have worked with Scrumban logic: enough structure to stay on course, enough flexibility to absorb the unexpected.

This constraint explains the separation between:

the MVP, which must prove the main flow;

the prospects for development, which contain strong ambitions but not essential to the first proof;

the 10/10 roadmap, which serves to harden the platform without pretending that everything is already finished.

The real risk was not only technical: it was wanting to make Notion, Supabase, Retool, Obsidian and Grafana at the same time. The quality constraint therefore forced us to reduce the scope, document the arbitrations and assume what remained outside the MVP.

?

?

?

Osionos ? Project file

the scope,

document the arbitrations and assume what remained outside the MVP.

?

?

?

Osionos ? Project file

29 / 101

the scope,
document the arbitrations and assume what remained outside the MVP.
?
?
?

Osionos ? Project file

29 / 101

In fact, I carried an important part of the role of product owner / project manager - framing of the vision, prioritization of the MVP, arbitration between functionalities, writing of the file and coordination with the technical constraints posed by the architectural profiles. Vadim and Roxana weighed a lot on architectural and quality requirements, in particular on the separation of services, security, reproducibility and testing strategy. Sergio carried out the back-end architecture and CI, and Daniel worked on the real-time foundations (WebSockets, game engine) which subsequently nourished the brick realtime of BaaS.

The working method is similar to a Scrumban: backlog and prioritization as in Scrum, more flexible execution like in Kanban. We kept short schedules at the start of each cycle, we followed the progress on a Kanban board, and we allowed ourselves to reorder without ceremony when the technical reality demanded it from us. This choice was adapted to the context: many unknowns techniques, a team that learns as it advances, and a scope that had to remain controllable despite the ambition of the product.

On the tools side, we deliberately separated real-time communication and project monitoring. For the communication, we used Discord as the main base (voice + written rooms by subject), WhatsApp for quick, off-topic exchanges, and Slack for some more formal channels. For monitoring of project, we went directly through GitHub Projects on the Univers42 organization: Kanban board, issues linked to PRs, milestones, all in the same place as the code.

The detailed rendering of the board (columns, milestones, issues) is reproduced in large format in the Annex section

es ? fig.9 at the end of the file.

Osionos ? Project file

31 / 101

opic exchanges, and Slack for some more formal channels. For monitoring of project, we went directly through GitHub Projects on the Univers42 organization: Kanban board, issues linked to PRs, milestones, all in the same place as the code.

The detailed rendering of the board (columns, milestones, issues) is reproduced in large format in the Annex section

es ? fig.9 at the end of the file.

Osionos ? Project file

31 / 101

opic exchanges, and Slack for some more formal channels. For monitoring of project, we went directly through GitHub Projects on the Univers42 organization: Kanban board, issues linked to PRs, milestones, all in the same place as the code.

The detailed rendering of the board (columns, milestones, issues) is reproduced in large format in the Annex section es ? fig.9 at the end of the file.

Osionos ? Project file

31 / 101

We also tried Notion at startup for documentation, and we finally abandoned it: that created two sources of truth (Notion on one side, the repo on the other), and at the slightest change of architecture the doc Notion was silently becoming false. We have therefore repatriated everything in this wiki, alongside the

code, so that PRs can correct the doc in the same gesture as the code they modify.

On the version control side, we worked in Git + GitHub with a model close to Git Flow: a branch protected hand which represents the stable state, a develop integration branch, branches feature/* for new features, fix/* for fixes and release/* for preparations version. On GitHub, we activated branch protection rules on main (and later on develop): no direct push, a mandatory pull request with at least one code review approved, and the green CI as a merge condition. To keep a readable history, we had also enforced commits in Conventional Commits format, controlled by local Git hooks (commit-msg , pre-commit) which rejected non-compliant messages and launched a quick lint before commit. This device ran for several months and worked properly ? it finished by being reduced when the team was reduced to two people, not because it was inefficient, but because the two of us wasted more time waiting for the formal review than correcting a poorly committed commit

formatted. We kept the hooks, we kept the PR on hand, and we agreed to be more pragmatic on the other branches.

B. TECHNICAL ENVIRONMENT

The technical environment can be summed up in one sentence: a Linux station, Docker as the only one runtime, VS Code as editor, and Make as control interface. The detail matters, because it is this homogeneity that allows each member of the team to have exactly the same platform, independently of their personal machine.

On the workstation side, we relied on the Linux ecosystem in all its diversity. The reference VM is a b2b VM under VirtualBox generated from Univers42/born2root (in the Born2beroot spirit), but in practice the team members ran the stack on Ubuntu, Debian, Kali Linux and Arch Linux without encountering any blocking problems. This is precisely what we were looking for: as long as Docker,

Docker Compose and Make are available, the rest of the stack does not make a difference. The main editor was VS Code, with some shared extensions (ESLint, Prettier, Docker, GitLens, Mermaid Preview) so that the code review is done in the same framework as the writing.

On the application stack side, we have four distinct but coherent stacks, which it is better to detail separately.

Front application (osionos) ? React 19, Vite 6, TypeScript strict, Zustand 5 for the state, @tanstack/react-virtual for virtualization, Playwright for end-to-end testing, ESLint + Prettier, all built and tested via apps/osionos/app/scripts/docker-run.sh in a container.

Front marketing (opposite-osiris) ? Astro 6, TypeScript, SCSS, @simplewebauthn/browser for passkeys, sanitize-html on the content side, and a container-only.mjs guardrail which purely and simply execution if we try to launch the project outside Docker.

BaaS and application services ? Kong 3.8 (DB-less, YAML versioned) as gateway, GoTrue 2.188 for auth, PostgREST 12 on PostgreSQL, NestJS for internal services (mongo-api, query-router, storage-router, permission-engine, session-service, schema-service, gdpr-service,

Osionos ? Project file

ter,

storage-router, permission-engine, session-service, schema-service, gdpr-service,

Osionos ? Project file

32 / 101

ter,
storage-router, permission-engine, session-service, schema-service, gdpr-service,
Osionos ? Project file
32 / 101

?desktop first?, not because we wanted to go against current trends, but because that our main target was professional users who would use Osionos on workstations work. This approach allowed us to focus on a rich and functional experience, without be limited by the constraints of a mobile design from the start. However, we ensured that the design is responsive, so that the application remains accessible on different types of devices. With Vadim, we created the models for the following screens (see fig.5 to fig.8) The models, although initially complicated to learn, allows us to save time in the long term if we realizes that the implementation choices do not meet the quality requirements.

Osionos ? Project file

36 / 101

would use Osionos on workstations

work. This approach allowed us to focus on a rich and functional experience, without be limited by the constraints of a mobile design from the start. However, we ensured that the design is responsive, so that the application remains accessible on different types of devices. With Vadim, we created the models for the following screens (see fig.5 to fig.8) The models, although initially complicated to learn, allows us to save time in the long term if we realizes that the implementation choices do not meet the quality requirements.

Osionos ? Project file

36 / 101

would use Osionos on workstations

work. This approach allowed us to focus on a rich and functional experience, without be limited by the constraints of a mobile design from the start. However, we ensured that the design is responsive, so that the application remains accessible on different types of devices. With Vadim, we created the models for the following screens (see fig.5 to fig.8) The models, although initially complicated to learn, allows us to save time in the long term if we realizes that the implementation choices do not meet the quality requirements.

Osionos ? Project file

36 / 101

GRAPHIC CHARTER

in order to ensure a coherent and consistent visual identity with the application standards, we have defined a clear graphic charter. The choice of colors and typography was guided by two imperatives. Respect for accessibility and the need for good readability for users who will read both webpage and application user interface.

Osionos ? Project file

39 / 101

GRAPHIC CHARTER

in order to ensure a coherent and consistent visual identity with the application standards, we have defined a clear graphic charter. The choice of colors and typography was guided by two imperatives. Respect for accessibility and the need for good readability for users who will read both webpage and application user interface.

Osionos ? Project file

39 / 101

As we can see, on lighthouse, we have very good results with a contrast of 7.5:1 for the normal texts and 4.5:1 for titles, which far exceeds WCAG 2.1 recommendations for accessibility. with a score of 100/100 in accessibility.

Osionos ? Project file

40 / 101

As we can see, on lightouse, we have very good results with a contrast of 7.5:1 for the normal texts and 4.5:1 for titles, which far exceeds WCAG 2.1 recommendations for accessibility. with a score of 100/100 in accessibility.

Osionos ? Project file

40 / 101

Screenshots of user interfaces

Calendar View ? built-in calendar for date/schedule pages, with monthly navigation and management of content blocks linked to each entry.

Home dashboard ? first thing you see when opening the app: a customizable dashboard with widgets created on the fly from the home page. This is where the user configures their space of work.

Osionos ? Project file

41 / 101

Screenshots of user interfaces

Calendar View ? built-in calendar for date/schedule pages, with monthly navigation and management of content blocks linked to each entry.

Home dashboard ? first thing you see when opening the app: a customizable dashboard with widgets created on the fly from the home page. This is where the user configures their space of work.

Osionos ? Project file

41 / 101

Entity-relationship diagram ? database diagram designed on Miro in advance of the development. It served as a reference throughout the project to structure the relationships between pages, blocks, workspaces and users.

Database rendering ? tabular view of an Osionos database, close to the Notion rendering. Each column is a configurable field, each row a record linked to a page.

Osionos ? Project file

42 / 101

Entity-relationship diagram ? database diagram designed on Miro in advance of the development. It served as a reference throughout the project to structure the relationships between pages, blocks, workspaces and users.

Database rendering ? tabular view of an Osionos database, close to the Notion rendering. Each column is a configurable field, each row a record linked to a page.

Osionos ? Project file

42 / 101

Project file translated into Japanese ? demonstration of the integrated translation functionality: this file has been automatically translated into Japanese from our internal rating system. A feature that we hadn't initially planned and that we slipped in because we could.

Email space ? messaging module integrated into the workspace, accessible directly from the sidebar. Allows you to manage communications without leaving the app.

Osionos ? Project file

43 / 101

Project file translated into Japanese ? demonstration of the integrated translation functionality: this file has been automatically translated into Japanese from our internal rating system. A feature that we hadn't initially planned and that we slipped in because we could.

Email space ? messaging module integrated into the workspace, accessible directly from the sidebar. Allows you to manage communications without leaving the app.

Osionos ? Project file

43 / 101

Login portal ? authentication page with login 42 OAuth2. Access is secure, tokens are managed on the BaaS side via GoTrue, and the secrets necessary for runtime are generated or retrieved via Vault rather than versioned.

Second Brain ? ?free note? view inspired by the concept of a digital second brain. A space without imposed structure where the user can think and organize freely with Osionos blocks.

Osionos ? Project file

44 / 101

Login portal ? authentication page with login 42 OAuth2. Access is secure, tokens are managed on the BaaS side via GoTrue, and the secrets necessary for runtime are generated or retrieved via Vault rather than versioned.

Second Brain ? ?free note? view inspired by the concept of a digital second brain. A space without imposed structure where the user can think and organize freely with Osionos blocks.

Osionos ? Project file

44 / 101

Front-end optimizations and measured results

On Osionos, front performance is not a bonus: the application displays long pages, blocks nested, visual databases, knowledge graph, context menus and settings panels. Without an explicit policy, the interface would become slow even before the user built a real workspace.

So I worked on four axes: not making what is not visible, not recalculating what is not changed, don't write to the backend on every keystroke, and don't load heavy libraries so much that they are not necessary. The optimizations below are taken from current code, not from a theoretical intention.

VIRTUALIZATION OF LONG BLOCKS

The read-only renderer uses `@tanstack/react-virtual` in `apps/osionos/app/src/widgets/page-renderer/ui/PageBlocksRenderer.tsx`. Virtualization does not activate immediately: it starts only above the threshold defined in `apps/osionos/app/src/entities/block/model/blockVirtualization.ts`, to avoid complicating the rendering of small pages.

The same component measures the actual offset with a `ResizeObserver`, because an Osionos page does not have

not lines of fixed height: a block can be a paragraph, an image, an inline base or a table.

CACHE AND STORAGE OF MARKDOWN RENDERING

Rendering blocks is expensive because the same text goes through the internal Markdown engine (markengine). To avoid parsing the same content several times, `apps/osionos/app/src/entities/block/ui/ReadOnlyBlock.tsx` uses a simple LRU cache, limited to 2000 entries.

```
// apps/osionos/app/src/widgets/page-renderer/ui/PageBlocksRenderer.tsx
const renderMeta = useMemo(() => createRootBlockRenderMeta(blocks), [blocks]);
const shouldVirtualize = blocks.length >= ROOT_BLOCK_VIRTUALIZATION_THRESHOLD;
const virtualizer = useVirtualizer({
  count: shouldVirtualize ? renderMeta.length: 0,
  getScrollElement: () => scrollElement,
  estimateSize: (index) => estimateBlockHeight(renderMeta[index]?.block ?? blocks[0]),
  getItemKey: (index) => renderMeta[index]?.block.id ?? index,
  overscan: ROOT_BLOCK_VIRTUALIZATION_OVERSCAN,
  scrollMargin,
});
```

Osionos ? Project file

```
Element: () => scrollElement,  
estimateSize: (index) => estimateBlockHeight(renderMeta[index]?.block ?? blocks[0]),  
getItemKey: (index) => renderMeta[index]?.block.id ?? index,  
overscan: ROOT_BLOCK_VIRTUALIZATION_OVERSCAN,  
scrollMargin,  
});
```

Osionos ? Project file

45 / 101

```
Element: () => scrollElement,  
estimateSize: (index) => estimateBlockHeight(renderMeta[index]?.block ?? blocks[0]),  
getItemKey: (index) => renderMeta[index]?.block.id ?? index,  
overscan: ROOT_BLOCK_VIRTUALIZATION_OVERSCAN,  
scrollMargin,  
});
```

Osionos ? Project file

45 / 101

The final component is also protected by `React.memo`, with a comparison targeted on the block, its index and its depth. The objective is not to put memo everywhere, but to protect the very nodes numerous.

DELAYED BACKUP AND ASYNCHRONOUS CALLS

Each keystroke in the editor does not trigger a network request. Content persistence is debounced at 400 ms in `apps/osionos/app/src/store/pageStore.persistence.ts`, and settings use the same principle via `apps/osionos/app/src/store/settings/settingsStoreUtils.ts`.

```
// apps/osionos/app/src/entities/block/ui/ReadOnlyBlock.tsx
const INLINE_MARKDOWN_CACHE_LIMIT = 2000;
const inlineMarkdownCache = new Map<string, React.ReactNode>();
function renderCachedInlineMarkdown(content: string): React.ReactNode {
  const cached = inlineMarkdownCache.get(content);
  if (cached !== undefined) {
    inlineMarkdownCache.delete(content);
    inlineMarkdownCache.set(content, cached);
    return cached;
  }
  const rendered = timed("renderInlineToReact", () => renderInlineToReact(content, {
    internalLinkRenderer: renderInternalPageLink,
  }));
  inlineMarkdownCache.set(content, rendered);
  if (inlineMarkdownCache.size > INLINE_MARKDOWN_CACHE_LIMIT) {
    const oldestKey = inlineMarkdownCache.keys().next().value;
    if (oldestKey !== undefined) inlineMarkdownCache.delete(oldestKey);
  }
  return rendered;
}
// apps/osionos/app/src/entities/block/ui/ReadOnlyBlock.tsx
function areReadOnlyBlockPropsEqual(previous: BlockProps, next: BlockProps): boolean {
  return(
    previous.block === next.block &&
    previous.index === next.index &&
    (previous.bulletDepth ?? 0) === (next.bulletDepth ?? 0) &&
    (previous.numberedDepth ?? 0) === (next.numberedDepth ?? 0)
  );
}
export const ReadOnlyBlock = React.memo(ReadOnlyBlockImpl, areReadOnlyBlockPropsEqual);
```

Osionos ? Project file

```
letDepth ?? 0) &&  
(previous.numberedDepth ?? 0) === (next.numberedDepth ?? 0)  
);  
}  
export const ReadOnlyBlock = React.memo(ReadOnlyBlockImpl, areReadOnlyBlockPropsEqual);  
Osionos ? Project file  
46 / 101
```



```
letDepth ?? 0) &&  
(previous.numberedDepth ?? 0) === (next.numberedDepth ?? 0)  
);  
}  
export const ReadOnlyBlock = React.memo(ReadOnlyBlockImpl, areReadOnlyBlockPropsEqual);  
Osonos ? Project file  
46 / 101
```

Page translation also shows a performance practice: each block is translated in a manner asynchronous, with a promise cache to avoid translating the same text twice in the same operation. It's in `apps/osionos/app/src/services/page-actions/index.ts`.

TARGETED DELAYED LOADING

Lazy loading exists, but you have to be precise: Mermaid is loaded dynamically by `apps/osionos/app/src/shared/ui/molecules/MermaidDiagram/MermaidDiagram.tsx`, and the subsystem of Embedded database (`notion-database-sys`) uses `React.lazy` in its component `object_database.tsx` for `DatabaseBlock`, `BlockHandle` and `PageModal`. On the other hand, `KaTeX` is imported statically into certain block components; I therefore do not present it as lazy-loaded in the current state of the code.

```
// apps/osionos/app/src/store/pageStore.persistence.ts
const _contentTimers = new Map<string, ReturnType<typeof setTimeout>>();
export function debouncePersistContent(pagelId: string) {
  const existing = _contentTimers.get(pagelId);
  if (existing) clearTimeout(existing);
  _contentTimers.set(pagelId, setTimeout(() => {
    _contentTimers.delete(pagelId);
    persistPageContent(pagelId);
  }, 400));
}

// apps/osionos/app/src/services/page-actions/index.ts
const cacheKey = `${targetLocale}\u0000${text}`;
const cached = cache.get(cacheKey);
if (cached) return cached;
const promise = (async () => {
  for (const translator of [
    () => translateWithConfiguredEndpoint(text, targetLocale, jwt),
    () => translateWithGooglePublicEndpoint(text, targetLocale),
    () => translateWithMyMemory(text, targetLocale),
  ]) {
    try {
      const translated = await translator();
      if (translated && !looksLikePrefixTranslation(translated, targetLocale)) {
        return translated;
      }
    } catch {
      // Try the next translation provider.
    }
  }
  return text;
})();
```

Osionos ? Project file

```
slated, targetLocale)) {  
  return translated;  
}  
} catch {  
  // Try the next translation provider.  
}  
}  
return text;  
})();  
Osionos ? Project file  
47 / 101
```

```
slated, targetLocale)) {  
    return translated;  
}  
} catch {  
    // Try the next translation provider.  
}  
}  
return text;  
})();  
Osionos ? Project file  
47 / 101
```

PERFORMANCE MEASUREMENT IN DEVELOPMENT

The performance is instrumented by `apps/osionos/app/src/shared/lib/perf/measure.ts` and connected from the React root in `apps/osionos/app/src/app/main.tsx`. In dev, any span greater than 4 ms emits a warning `[perf]`, which forces you to see the small costs that accumulate.

BUILD, BUNDLE AND SEO

The osionos private application is a Vite SPA: it is not designed for public referencing. `Sound index.html` keeps the necessary basics (lang, viewport, title), but the product's SEO strategy is carried by the Astro opposite-osiris site, which renders static HTML and defines tags description, color-scheme, favicon, CSP and preconnect in `apps/opposite-osiris/src/layouts/Layout.astro`.

```
// apps/osionos/app/src/shared/ui/molecules/MermaidDiagram/MermaidDiagram.tsx
```

```
let mermaidInitialized = false;
```

```
let mermaidPromise: Promise<typeof import("mermaid").default> | null = null;
```

```
function loadMermaid() {
```

```
  mermaidPromise ??= import("mermaid").then((module) => module.default);
```

```
  return mermaidPromise;
```

```
}
```

```
async function ensureMermaidInitialized() {
```

```
  const mermaid = await loadMermaid();
```

```
  if (mermaidInitialized) return mermaid;
```

```
  mermaid.initialize({
```

```
    startOnLoad: false,
```

```
    securityLevel: "strict",
```

```
    theme: "default",
```

```
  });
```

```
  mermaidInitialized = true;
```

```
  return mermaid;
```

```
}
```

```
// apps/osionos/app/src/app/main.tsx
```

```
createRoot(root).render(
```

```
  <StrictMode>
```

```
    <Profiler id="App" onRender={recordReactCommit}>
```

```
      <App />
```

```
    </Profile>
```

```
  </StrictMode>,
```

```
);
```

```
// apps/osionos/app/src/shared/lib/perf/measure.ts
```

```
const WARN_THRESHOLD_MS = 4;
```

```
function warnIfSlow(name: string, durationMs: number) {
```

```
  if (durationMs > WARN_THRESHOLD_MS) {
```

```
    console.warn(`[perf] ${name}: ${durationMs.toFixed(1)}ms`);
```

```
  }
```

```
}
```

Osionos ? Project file

```
s > WARN_THRESHOLD_MS) {  
  console.warn(`[perf] ${name}: ${durationMs.toFixed(1)}ms`);  
}  
}
```

Osionos ? Project file

48 / 101

```
s > WARN_THRESHOLD_MS) {  
  console.warn(`[perf] ${name}: ${durationMs.toFixed(1)}ms`);  
}  
}
```

Osionos ? Project file

48 / 101

The Lighthouse capture available in the folder was taken at <https://localhost:4322/>, so on the Prismatica marketing site, not on the private app. It shows: Performance 85, Accessibility 100, Best Practices 96, SEO 100, with FCP 1.6 s, LCP 1.8 s, Total Blocking Time 0 ms and CLS 0. These results confirm two choices: marketing is of course Astro for SEO and performance perceived, while the private React/Vite application assumes a different logic, focused on interaction rich, local persistence and productivity.

Code snippets, static user interfaces (React / SCSS)

A. MINIMUM ORGANIZATION OF THE FRONT-END PROJECT

The Osionos front is not organized as a simple collection of React components. It follows a organization close to Feature-Sliced Design: the business elements live in entities, the interactions in features, visible assemblies in widgets, orchestration in app, and reusable components in shared .

```
<!-- apps/opposite-osiris/src/layouts/Layout.astro -->
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<meta name="color-scheme" content="light dark" />
<meta name="description" content={description} />
<meta http-equiv="Content-Security-Policy" content={activeCsp} />
<link rel="preconnect" href="https://fonts.googleapis.com" />
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
<title>{title}</title>
```

File

Role in Osionos

Verified examples

`apps/osionos/app/`

`src/app`

Entry point, global styles,

main shell

`main.tsx`, `App.tsx`, CSS tokens

`apps/osionos/app/`

`src/entities`

Displayable business objects

page, block, user

Osionos ? Project file

49 / 101

n />

<title>{title}</title>

File

Role in Osionos

Verified examples

apps/osionos/app/

src/app

Entry point, global styles,

main shell

main.tsx, App.tsx, CSS tokens

apps/osionos/app/

src/entities

Displayable business objects

page, block, user

Osionos ? Project file

49 / 101

n />

<title>{title}</title>

File

Role in Osionos

Verified examples

apps/osionos/app/

src/app

Entry point, global styles,

main shell

main.tsx, App.tsx, CSS tokens

apps/osionos/app/

src/entities

Displayable business objects

page, block, user

Osionos ? Project file

49 / 101

The entry point is deliberately very thin. He mounts React 19, activates StrictMode, plugs it in Profiler, then let App.tsx assemble the shell.

App.tsx is the application shell: it initializes the session, applies the theme, chooses between debug mode, the Prismatic handoff screen, then the main layout with sidebar, content, settings and notifications.

File

Role in Osionos

Verified examples

apps/osionos/app/

src/features

User interactions

auth, block editor, page management, slash

commands, settings

apps/osionos/app/

src/widgets

Compound UI Areas

sidebar, page renderer, database view, channel

messages, graph home

apps/osionos/app/

src/shared

Client API, hooks, UI primitives,

config, performance

api/client.ts, Modal.tsx,

Dropdown.tsx, measure.ts

apps/osionos/app/

src/store

Zustand binds and persistence

pages, database, settings

apps/osionos/app/

src/services

Application actions outside

component

actions page, realtime messages

// apps/osionos/app/src/app/main.tsx

```
import { Profiler, StrictMode } from "react";
```

```
import { createRoot } from "react-dom/client";
```

```
import App from "../App.tsx";
```

```
import { recordReactCommit } from '@shared/lib/perf/measure';
```

```
import './styles/global.css';
```

```
const root = document.getElementById("root");
```

```
if (root) {
```

```
  createRoot(root).render(
```

```
    <StrictMode>
```

```
    <Profiler id="App" onRender={recordReactCommit}>
```

```
    <App />
```

```
    </Profile>
```

```
  </StrictMode>,
```

```
);
```

```
}
```

Osionos ? Project file

50 / 101

```
</StrictMode>,
```

```
);
```

```
}
```

Osionos ? Project file

50 / 101

```
</StrictMode>,
```

```
);
```

```
}
```

Osionos ? Project file

50 / 101

The front stack is visible in apps/osionos/app/package.json: React 19, Vite 6, TypeScript, Zustand, Playwright, lucid-react, @tanstack/react-virtual, Mermaid, KaTeX, Leaflet, Recharts. The scripts all go through scripts/docker-run.sh , which forces the same build and test environment for everyone.

```
// apps/osionos/app/src/app/App.tsx
return(
  <div
    data-testid="app-shell"
    className="relative flex h-screen w-screen overflow-hidden bg-[var(--osio-bg-page)]"
  >
    <Sidebar
      onOpenSettings={() => setSettingsOpen(true)}
      onOpenHome={() =>
        usePageStore.setState({
          activePage: null,
          showTrash: false,
          navigationPath: [],
        })
      }
      onOpenTrash={() =>
        usePageStore.setState({
          activePage: null,
          showTrash: true,
          navigationPath: [],
        })
      }
    />
    <SidebarTrigger />
    <main className="flex-1 flex min-w-0 overflow-hidden relative">
      <MainContent />
    </main>
    <WorkspaceThemePanel />
    {settingsOpen && <SettingsCenter initialTab="general" onClose={() =>
      setSettingsOpen(false)} />}
    <ToastViewport />
  </div>
);
```

```
// apps/osionos/app/package.json
```

```
{
  "scripts": {
    "build": "bash scripts/docker-run.sh build",
    "typecheck": "bash scripts/docker-run.sh typecheck",
    "lint": "bash scripts/docker-run.sh lint",
    "test:e2e": "bash scripts/docker-run.sh test-e2e",
    "test:canvas": "bash scripts/docker-run.sh test-canvas",
    "test:bridge": "bash scripts/docker-run.sh test-bridge",
    "test:quality": "bash scripts/docker-run.sh quality"
  }
}
```

Osionos ? Project file

y"

}

}

Osionos ? Project file

51 / 101

y"

}

}

Osionos ? Project file

51 / 101

B. CODE Snippet 1: STATIC AND ACCESSIBLE LOGIN PORTAL

The connection portal visible in the captures does not live directly in the osionos private SPA. He is rendered on the Astro side in opposite-osiris , because this part must be fast, indexable and accessible even before the user opens their workspace. This is an important choice: the page public is static and SEO-friendly; The private React app starts after secure handoff.

The apps/opposite-osiris/src/components/ui/Portal.astro component shows this attention to accessibility: dialog, title linked by aria-labelledby, labels associated with fields, aria-messages live, named buttons, explicit consents and Turnstile anti-abuse zone.

Client-side validation is carried by apps/opposite-osiris/src/hooks/useAuth.ts. She checks the email, complexity of the password, the anti-abuse token, then calls the gateway with credentials: 'include' and a controlled retry on the responses 429.

```
<!-- apps/opposite-osiris/src/components/ui/Portal.astro -->
<dialog
  id="portal"
  class={`portal portal--${quick ? 'quick': 'start'}`}
  aria-labelledby="portal-title"
  data-default-mode={quick ? 'connect': 'start'}
>
  <h2 id="portal-title" class="visually-hidden">Prismatica workspace portal</h2>
  <button class="portal__close" type="button" aria-label="Close portal" data-close-portal>x</button>
  <section class="portal__panel portal__panel--login" aria-label="Secure connection panel">
    <form class="portal-login" novalidate>
      <label for="portal-email">Email <span aria-hidden="true">*</span></label>
      <input id="portal-email" name="email" type="email" autocomplete="email"
inputmode="email" required />
      <p id="portal-email-inline-error" class="field-validation-message" aria-live="polite">
        We verify the email format before sending it.
      </p>
      <div class="turnstile-box" data-turnstile-widget aria-label="Anti-abuse check"></div>
      <output id="portal-error-msg" class="portal-error" role="status" aria-live="polite" aria-atomic="true"></output>
    </form>
  </section>
</dialog>
```

Osionos ? Project file

v>

<output id="portal-error-msg" class="portal-error" role="status" aria-live="polite" aria-atomic="true"></output>

</form>

</section>

</dialog>

Osionos ? Project file

52 / 101

v>

```
<output id="portal-error-msg" class="portal-error" role="status" aria-  
live="polite" aria-atomic="true"></output>
```

```
</form>
```

```
</section>
```

```
</dialog>
```

Osionos ? Project file

52 / 101

Accessibility is not limited to the portal. In the React application, common primitives also carry keyboard behaviors: `apps/osionos/app/src/shared/ui/primitives/Modal.tsx` manages `role="dialog"` , `aria-modal` , `Escape` , `initial focus`, `focus trap` and `focus restoration`; `apps/osionos/app/src/shared/ui/primitives/Dropdown.tsx` implements `combobox/listbox` with navigation keyboard.

Finally, responsive design relies on CSS tokens and fluid values rather than a stack of breakpoints. `apps/osionos/app/src/pages/notion-page/ui/notionPage.css` uses `clamp()` to keep comfortable reading on small and large screens.

```
// apps/opposite-osiris/src/hooks/useAuth.ts
```

```
export const RFC_5322_EMAIL_REGEX = new RegExp(String.raw`${EMAIL_LOCAL_PART}@(?:${EMAIL_DOMAIN_LABEL}\.)+[A-Za-z]{2,63}$`);
```

```
export const STRONG_PASSWORD_REGEX = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^\A-Za-z0-9]).{8,}$/;
```

```
function validationMessage(request: AuthRequest, mode: AuthMode): string | null {
```

```
  if (!validateEmail(request.email)) return 'Use a valid email address.';
```

```
  if (mode === 'register' && !validatePassword(request.password)) {
```

```
    return 'Password must be at least 8 characters and include uppercase, lowercase, number, and symbol.';
```

```
  }
```

```
  if (mode === 'login' && request.password.length === 0) return 'Enter your password.';
```

```
  if (!request.turnstileToken) return 'Complete the anti-abuse check.';
```

```
  return null;
```

```
}
```

```
// apps/osionos/app/src/shared/ui/primitives/Modal.tsx
```

```
<div
```

```
  ref={dialogRef}
```

```
  role="dialog"
```

```
  aria-modal="true"
```

```
  aria-labelledby={title ? titleId: undefined}
```

```
  aria-describedby={description ? descriptionId: undefined}
```

```
  tabIndex=-1}
```

```
>
```

```
  {title ? <h2 id={titleId} className="sr-only">{title}</h2>: null}
```

```
  {description ? <p id={descriptionId} className="sr-only">{description}</p>: null}
```

```
  {children}
```

```
</div>
```

Osionos ? Project file

53 / 101

```
ed}
tabIndex={-1}
>
{title ? <h2 id={titleId} className="sr-only">{title}</h2>: null}
{description? <p id={descriptionId} className="sr-only">{description}</p>: null}
{children}
</div>
```

Osionos ? Project file

53 / 101

```
ed}
tabIndex={-1}
>
{title ? <h2 id={titleId} className="sr-only">{title}</h2>: null}
{description? <p id={descriptionId} className="sr-only">{description}</p>: null}
{children}
</div>
Osionos ? Project file
53 / 101
```

Code snippets, dynamic part

A. AUTHENTICATION: PRISMATICA SESSION, SECURE BRIDGE AND OFFLINE FALLBACK

Authentication on the Osionos side is not just a React form. The real flow is in two stages: the Astro site (opposite-osiris) authenticates the user, then the osionos application consumes a signed bridge session. If no bridge is available and offline mode is authorized, the application starts with seeded data to allow local development.

In `apps/osionos/app/src/features/auth/model/userStore.helpers.ts`, the bridge token is read from the URL, sent to the API, then immediately removed from the address bar to avoid it remaining in visible history.

The Zustand `apps/osionos/app/src/features/auth/model/useUserStore.ts` store centralizes the user state, sessions, active workspaces and offline fallback. It contains a safeguard against the double call of `init()` in `StrictMode`, which is essential with React 19.

```
/* apps/osionos/app/src/pages/notion-page/ui/notionPage.css */
.osionos-page-header,
.osionos-page-properties,
.osionos-page-body {
  max-width: var(--page-content-max-width, 900px);
  width: 100%;
  min-width: 0;
  margin-left: auto;
  margin-right: auto;
  padding-left: var(--page-content-padding-inline, clamp(16px, 11%, 96px));
  padding-right: var(--page-content-padding-inline, clamp(16px, 11%, 96px));
}

// apps/osionos/app/src/features/auth/model/userStore.helpers.ts
export async function consumeBridgeSessionFromLocation(): Promise<BridgeSessionImport | null> {
  const token = bridgeTokenFromLocation();
  if (!token || !API_BASE) return null;
  const response = await fetch(`${API_BASE}/api/auth/bridge/consume`, {
    method: 'POST',
    headers: { Accept: 'application/json', 'Content-Type': 'application/json' },
    credentials: 'include',
    body: JSON.stringify({ token }),
  });
  if (!response.ok) throw new Error('Bridge session could not be imported.');
```

```
const payload = await response.json() as BridgeSessionImport;
clearBridgeTokenFromLocation();
return payload;
}
```

Osionos ? Project file

```
t be imported.');
```

```
const payload = await response.json() as BridgeSessionImport;
```

```
clearBridgeTokenFromLocation();
```

```
return payload;
```

```
}
```

Osionos ? Project file

54 / 101


```
t be imported.');
```

```
const payload = await response.json() as BridgeSessionImport;
```

```
clearBridgeTokenFromLocation();
```

```
return payload;
```

```
}
```

Osionos ? Project file

54 / 101

The common API client apps/osionos/app/src/shared/api/client.ts adds the JWT only when it exists and transforms HTTP errors into typed ApiErrors. It's a small layer, but it prevents each component rebuilds its own fetch logic.

```
// apps/osionos/app/src/features/auth/model/useUserStore.ts
```

```
let _initInProgress = false;
export const useUserStore = create<UserStore>((set, get) => ({
  personas: uniquePersonas([...INITIAL_PERSONAS.map(p => ({ ...p })), ...readPersistedPe
rsonas()])),
  sessions: {},
  activeUserId: "",
  initialized: false,
  loading: false,
  error: null,
  init:async() => {
    if (get().initialized || _initInProgress) return;
    _initInProgress = true;
    set({ loading: true, error: null });
    try {
      set(await resolveInitialState());
    } catch {
      set(bridgeOnlyMode() ? bridgeSessionRequiredState(): offlineState());
    } finally {
      _initInProgress = false;
    }
  },
  }));
```

```
// apps/osionos/app/src/shared/api/client.ts
```

```
async function request<T>(method: string, path: string, body?: unknown, jwt?: string): P
romise<T> {
  if (!API_BASE) throw new Error("VITE_API_URL is not configured.");
  const headers: Record<string, string> = { 'Content-Type': 'application/json' };
  if (jwt) headers['Authorization'] = `Bearer ${jwt}`;
  const res = await fetch(`${API_BASE}${path}`, {
    methods,
    headers,
    body: body == null ? undefined: JSON.stringify(body),
  });
  if (!res.ok) {
    const errorBody = await res.json().catch(() => null) as ApiErrorBody | null;
    throw new ApiError(errorBody?.error ?? errorBody?.message ?? `${method} ${path} ? $
${res.status} ${res.statusText}`, res.status);
  }
  if (res.status === 204) return undefined as T;
  return res.json() as Promise<T>;
}
```

Osionos ? Project file

```
04) return undefined as T;  
return res.json() as Promise<T>;  
}
```

Osionos ? Project file

55 / 101

```
04) return undefined as T;  
return res.json() as Promise<T>;  
}
```

Osionos ? Project file

55 / 101

B. DATA RECOVERY: WORKSPACES, PAGES AND FULL CONTENT

On the product side, dynamic retrieval covers workspaces, pages and complete content of one page. This is the central flow of Osionos: the user opens a space, chooses a page, then The application only loads what is necessary to view or edit this page.

In `apps/osionos/app/src/store/pageStore.actions.ts`, `fetchPages` first checks the JWT usable by the pages API, then the current access context. If the user does not belong to the requested workspace, the query doesn't even go away. This front verification is not the final security (it remains on the BaaS/RLS side), but it avoids bad UX and reduces unnecessary calls.

The entire content of a page is loaded on demand. `apps/osionos/app/src/widgets/page-renderer/ui/MainContent.tsx` only fetches if the active page is a real page, the JWT exists and the content is not already in the store.

```
// apps/osionos/app/src/store/pageStore.actions.ts
export function createFetchPages(set: SetFn, get: GetFn) {
  return async(workspaceId: string, jwt: string) => {
    const pageJwt = pageApiJwtFromSessionToken(jwt);
    if (!pageJwt) return;
    const context = getCurrentPageAccessContext();
    if (context && !context.workspaceIds.includes(workspaceId)) return;
    if (get().loadingIds.has(workspaceId)) return;
    set((s) => ({ loadingIds: new Set([...s.loadingIds, workspaceId]) }));
    try {
      const data = await api.get<PageEntry[]>(
        `/api/pages/all?workspaceId=${workspaceId}`,
        pageJwt,
      );
      set((s) => ({
        ...derivePageState({
          ...s.pages,
          [workspaceId]: mergeWorkspacePages(s.pages[workspaceId], data),
        }, s.pageIdsByWorkspace),
        loadingIds: new Set([...s.loadingIds].filter((id) => id !== workspaceId)),
      }));
      savePagesCache(get().pages, workspaceId);
    } catch {
      set((s) => ({ loadingIds: new Set([...s.loadingIds].filter((id) => id !== workspaceId)) }));
    }
  };
}
```

```
// apps/osionos/app/src/widgets/page-renderer/ui/MainContent.tsx
useEffect(() => {
  if (!activePage || activePage?.kind !== "page" || !jwt) return;
  const page = pageById(activePage.id);
  if (!page) {
    fetchPageContent(activePage.id, jwt);
  }
}, [activePage, jwt, pageById, fetchPageContent]);
```

Osionos ? Project file

```
wt, pageByld, fetchPageContent]);
```

Osionos ? Project file

56 / 101

wt, pageByld, fetchPageContent]);

Osionos ? Project file

56 / 101

The function called `storeSide` then checks that the page exists, that the user can read it, then merges API revenue fields into local state.

C. BUSINESS CRITICAL ACTIONS: ARCHIVE, DELETE, LOCK, TRANSLATE, RESTORE

In Osionos, the critical business actions are: archive a page, delete a page, duplicate a page, change its implicit permissions, lock editing, translate its content, or restore a version. These are actions visible to the user, but they also modify the local state, persistence and sometimes the descendants of a page.

Before performing a dangerous action, `apps/osionos/app/src/features/page-management/ui/PageOptionsMenu.tsx` checks the local access context via `canDeletePage` or `canDuplicatePage` and then calls the blind. This check does not replace the backend; it protects the interface and avoids propose inconsistent actions.

```
// apps/osionos/app/src/store/pageStore.actions.ts
export function createFetchPageContent(set: SetFn, get: GetFn) {
  return async(pagId: string, jwt: string) => {
    const pageJwt = pageApiJwtFromSessionToken(jwt);
    if (!pageJwt || !isPersistedPagId(pagId)) return;
    const page = get().pageById(pagId);
    const context = getCurrentPageAccessContext();
    if (!page || !canReadPage(page, context)) return;
    try {
      const fullPage = await api.get<PageEntry>(`/api/pages/${pagId}`, pageJwt);
      if (!fullPage) return;
      set((s) => ({
        ...derivePageState(updatePageInState(s.pages, pagId, (p) => ({
          ...p,
          content: fullPage.content ?? p.content,
          title: fullPage.title ?? p.title,
          icon: fullPage.icon ?? p.icon,
          cover: fullPage.cover ?? p.cover,
          updatedAt: fullPage.updatedAt ?? p.updatedAt,
        })), s.pageIdsByWorkspace),
      }));
      savePagesCache(get().pages, page.workspaceId);
    } catch(err) {
      console.warn("[pageStore] fetchPageContent failed:", pagId, err);
    }
  };
}
```

Osionos ? Project file


```
IdsByWorkspace),
    });
    savePagesCache(get().pages, page.workspaceId);
  } catch(err) {
    console.warn("[pageStore] fetchPageContent failed:", pageId, err);
  }
};
}
Osionos ? Project file
57 / 101
```

```
IdsByWorkspace),  
    });  
    savePagesCache(get().pages, page.workspaceId);  
  } catch(err) {  
    console.warn("[pageStore] fetchPageContent failed:", pageId, err);  
  }  
};  
}
```

Osionos ? Project file

57 / 101

Front access rules are centralized in `apps/osionos/app/src/shared/lib/auth/pageAccess.ts`, at instead of being copied into each component.

Archiving clearly shows the business logic: we patch the backend when a JWT exists, then we update locally the page and all its descendants, also cleaning recent pages. It's in `apps/osionos/app/src/store/pageStore.actions.ts`.

```
// apps/osionos/app/src/features/page-management/ui/PageOptionsMenu.tsx
const handleDuplicateClick = async(e:React.MouseEvent) => {
  e.stopPropagation();
  setIsMenuOpen(false);
  if (!workspaceId) return;
  if (!currentPage || !canDuplicatePage(currentPage, getCurrentPageAccessContext())) return;
  try {
    await duplicatePage(pageId, workspaceId);
  } catch(err) {
    console.error("[PageOptionsMenu] Failed to duplicate page", err);
  }
};
const handleConfirmDelete = async () => {
  if (!workspaceId) return;
  if (!currentPage || !canDeletePage(currentPage, getCurrentPageAccessContext())) return;
  await deletePage(pageId, workspaceId, jwt ?? "");
  redirectIfAffectedPageChanged();
};
// apps/osionos/app/src/shared/lib/auth/pageAccess.ts
export function canReadPage(page: PageEntry, context: PageAccessContext | null):
boolean {
  if (!context || !hasWorkspaceAccess(page, context)) return false;
  const visibility = normalizePageVisibility(page.visibility);
  if (visibility === "public") return true;
  if (visibility === "shared") return true;
  if (page.ownerId && page.ownerId === context.userId) return true;
  if (isLegacyPage(page)) return true;
  return getCollaboratorRole(page, context.userId) !== null;
}
export function canEditPage(page: PageEntry, context: PageAccessContext | null):
boolean {
  if (!context || !hasWorkspaceAccess(page, context)) return false;
  if (context.sharedWorkspaceIds.includes(page.workspaceId)) return true;
  if (page.ownerId && page.ownerId === context.userId) return true;
  if (isLegacyPage(page)) return true;
  const collaboratorRole = getCollaborativeRole(page, context.userId);
  return collaboratorRole === "editor" || collaboratorRole === "owner";
}
```

Osionos ? Project file

```
laboratorRole === "editor" || collaboratorRole === "owner";  
}
```

Osionos ? Project file

58 / 101

```
laboratorRole === "editor" || collaboratorRole === "owner";  
}
```

Osionos ? Project file

58 / 101

More advanced page actions are grouped in `apps/osionos/app/src/entities/page/model/usePageActions.ts`. This hook manages the word counter, automatic versions, translation, import/export, notifications, presentation mode and page locking.

```
// apps/osionos/app/src/store/pageStore.actions.ts
export function createArchivePage(set: SetFn, get: GetFn) {
  return async(pagelId: string, workspaceId: string, jwt: string) => {
    const page = get().pageById(pagelId);
    const context = getCurrentPageAccessContext();
    if (!page || !canDeletePage(page, context)) return;
    const archivedAt = new Date().toISOString();
    const pageJwt = pageApiJwtFromSessionToken(jwt);
    if (pageJwt && isPersistedPagelId(pagelId)) {
      try {
        await api.patch(`/api/pages/${pagelId}`, { archivedAt }, pageJwt);
      } catch {
        /* silent */
      }
    }
    set((s) => {
      const wsPages = s.pages[workspaceId] ?? [];
      const descendantIds = getAllDescendantIds(wsPages, pagelId);
      const archivedIds = new Set([pagelId, ...descendantIds]);
      const newRecents = s.recents.filter((r) => !archivedIds.has(r.id));
      const pages = {
        ...s.pages,
        [workspaceId]: wsPages.map((p) => archivedIds.has(p._id) ? { ...p,
          archivedAt } : p),
      };
      return { ...derivePageState(pages, s.pagelIdsByWorkspace), recents: newRecents };
    });
    savePagesCache(get().pages, workspaceId);
  };
}
```

Osionos ? Project file

```
ages,  
[workspaceId]: wsPages.map((p) => archivedIds.has(p._id) ? { ...p,  
  archivedAt } : p),  
};  
return { ...derivePageState(pages, s.pageIdsByWorkspace), recents: newRecents };  
});  
savePagesCache(get().pages, workspaceId);  
};  
}
```

Osionos ? Project file

59 / 101

```
ages,  
[workspaceId]: wsPages.map((p) => archivedIds.has(p._id) ? { ...p,  
  archivedAt } : p),  
};  
return { ...derivePageState(pages, s.pageIdsByWorkspace), recents: newRecents };  
});  
savePagesCache(get().pages, workspaceId);  
};  
}
```

Osionos ? Project file

59 / 101

Accessibility, front-line security and measurable quality

INTEGRATED COMPONENT ACCESSIBILITY

Accessibility is visible at several levels of the code: an avoidance link on the Astro page, buttons named, tabs with aria-selected, breadcrumbs with aria-current, an editor contentEditable announced as multiline textbox, and modals with focus trap.

```
// apps/osionos/app/src/entities/page/model/usePageActions.ts
const toggleLock = useCallback(
  () => updatePageSetting(
    { locked: !config.locked },
    'lock_page',
    config.locked? 'Page unlocked': 'Page locked',
  ),
  [config.locked, updatePageSetting],
);

const translate = useCallback(async (targetLocale = translateLocale) => {
  if (!page || !pageId) return;
  const label = translationLabel(targetLocale);
  await snapshot(`Before translation to ${label}`);
  const translated = await translatePage(page, jwt ?? undefined, targetLocale);
  if (translated.title) updatePageTitle(pageId, translated.title);
  if (translated.content) updatePageContent(pageId, translated.content);
  await logAction('translate', `Page translated to ${label}`, { targetLocale });
}, [jwt, logAction, page, pageId, snapshot, translateLocale, updatePageContent, updatePageTitle]);

<!-- apps/opposite-osiris/src/pages/index.astro -->
<a href="#main-content" class="skip-link">Skip to main content</a>
<div aria-live="polite" aria-atomic="true" class="visually-hidden" id="global-announcer"></div>
<main id="main-content" class="swipe-stack" data-swipe-stack>
  ...
</main>
// apps/osionos/app/src/widgets/sidebar/ui/SidebarTopNav.tsx
<div role="tablist" aria-label="Sidebar navigation">
  {tabs.map((tab) => (
    <button
      key={tab.id}
      type="button"
      role="tab"
      aria-selected={tab.active}
      aria-label={tab.label}
      title={tab.label}
    >
      <span className="flex shrink-0 items-center opacity-80">{tab.icon}</span>
      <span className={tab.active ? 'ml-1.5 truncate': 'sr-only'}>{tab.label}</span>
    </button>
  ))}
</div>
```

Osionos ? Project file

abel}

</button>

)))

</div>

Osionos ? Project file

60 / 101

abel}
</button>
)}}
</div>
Osionos ? Project file
60 / 101

TERMINATION AND NAVIGATION SAFETY

The front should never be presented as the final layer of security: the real barrier remains aside BaaS (Kong, JWT, PostgREST, RLS, owner_id). On the other hand, the front reduces the risk upon rendering. The

Markdown engine escapes HTML text, filters dangerous URL schemes and adds rel="noopener noreferrer" on external links.

```
// apps/osionos/app/src/components/blocks/EditableContent.tsx
```

```
<div
```

```
  ref={ref}
```

```
  role="textbox"
```

```
  aria-multiline="true"
```

```
  tabIndex={0}
```

```
  contentEditable
```

```
  suppressContentEditableWarning
```

```
  spellCheck
```

```
  data-placeholder={hasFocus ? placeholder: ""}
```

```
  onInput={handleInput}
```

```
  onKeyDown={handleKeyDown}
```

```
  onPaste={handlePaste}
```

```
// apps/osionos/app/src/shared/lib/markengine/renderCore.ts
```

```
export function escapeHtml(value: string): string {
```

```
  return value.replaceAll(HTML_ESCAPE_PATTERN, (char) => HTML_ESCAPE_MAP[char]);
```

```
}
```

```
export function sanitizeUrl(value: string): string {
```

```
  const normalized = stripUrlControlAndSpaceChars(value.trim());
```

```
  const schemeMatch = /^[a-z][a-z\d+.-]*:/i.exec(normalized);
```

```
  if (!schemeMatch) return value.trim();
```

```
  const scheme = schemeMatch[1].toLowerCase();
```

```
  if (scheme === "http" || scheme === "https" || scheme === "mailto" || scheme ===
```

```
  "tel") {
```

```
    return value.trim();
```

```
  }
```

```
  return "";
```

```
}
```

```
// apps/osionos/app/src/shared/lib/markengine/markdown/renderers/inlineHtml.ts
```

```
function renderLink(node: Extract<InlineNode, { type: "link" }>, options: ResolvedInline
```

```
HtmlOptions): string {
```

```
  const href = sanitizeUrl(node.href);
```

```
  const attrs = [
```

```
    `href="${esc(href || "#")}"`,
```

```
    options.externalLinks && isExternalUrl(href) ? `target="_blank" rel="noopener
```

```
    noreferrer": "",
```

```
  ].filter(Boolean).join(" ");
```

```
  return `<a ${attrs}>${renderChildren(node.children, options)}</a>`;
```

```
}
```

Osionos ? Project file

```
renderChildren(node.children, options)}</a>`;
}
```

Osionos ? Project file

61 / 101

```
renderChildren(node.children, options)}</a>`;
```

```
}
```

Osionos ? Project file

61 / 101

The syntax highlighting component `apps/osionos/app/src/shared/ui/molecules/CodeSyntaxHighlight/CodeSyntaxHighlight.tsx` uses `dangerouslySetInnerHTML`, but only after escaping manual for unknown languages and after `highlight.js` for registered languages. Finally, the Astro public site applies strict CSP in production in `apps/opposite-osiris/src/layouts/Layout.astro`, with `object-src 'none'`, `base-uri 'self'`, Trusted Types and `require-trusted-types-for 'script'`.

SUMMARY OF WHAT THE FRONT SHOWS

This front chapter therefore shows several concrete things that I have achieved or integrated: a modular React architecture, a static access portal accessible on the Astro side, a bridge session secure between Prismatica and Osionos, Zustand stores with offline fallback, page recovery asynchronous, protected business actions, virtualization of long pages, debounce of persistence, an instrumented Markdown engine, and a clear separation between public SEO (Astro) and rich private application (React/Vite).

CHAPTER 4. Personal achievements, back-end

This chapter presents the server part that I actually built or integrated. The back end is not a single monolithic server: it is a platform made up of specialized bricks. Kong plays the role of gateway, GoTrue manages authentication, PostgREST exposes PostgreSQL in REST, MongoDB serves the documentary data, MinIO stores the files, realtime-agnostic broadcasts the changes, and NestJS services carry the logic that we control directly: `mongo-api`, `query-router`, `adapter-registry`, `schema-service`, `permission-engine`, `storage-router`, `session-service`, `gdpr-service`, `log-service`, `email-service`, `newsletter-service`, `analytics-service` and `ai-service`.

```
// apps/osionos/app/src/shared/ui/molecules/CodeSyntaxHighlight/CodeSyntaxHighlight.tsx
```

```
function escapeHtml(value: string) {
  return value
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;")
    .replaceAll("'", "&quot;");
}
if (!hljs.getLanguage(normalized)) {
  return escapeHtml(code);
}
```

```
<!-- apps/opposite-osiris/src/layouts/Layout.astro -->
```

```
const productionCsp = [
  "default-src 'self'",
  "base-uri 'self'",
  "object-src 'none'",
  "form-action 'self'",
  "script-src 'self' https://challenges.cloudflare.com",
  "trusted-types prismatic-static-markup",
  "require-trusted-types-for 'script'",
].join('; ');
```

Osionos ? Project file

```
d-types prismatic-static-markup",  
  "require-trusted-types-for 'script'",  
].join('; ');
```

Osionos ? Project file

62 / 101


```
d-types prismatic-static-markup",  
  "require-trusted-types-for 'script'",  
].join('; ');
```

Osionos ? Project file

62 / 101

Kong is also the point where the identity becomes usable by internal services. The setup kong.yml checks JWT, applies key-auth, rate limiting, payload limits, CORS, headers security, then injects trusted headers (X-User-Id, X-User-Email, X-User-Role). The NestJS services therefore do not each revalidate the JWT: they read the identity already validated by the gateway.

B. CONCEPTUAL DATA DIAGRAM

The difficulty of the project is that there is not a single database. There is a relational base for identity, roles, permissions, records and structured data; there is an Osionos model workspace/page oriented; and there is a document/multi-engine plan for dynamic collections and the databases saved by the user.

The first diagram represents the BaaS core: users, demo content, roles, policies, saved databases, created schemas and storage objects.

```
# apps/baas/mini-baas-infra/docker/services/kong/conf/kong.yml
```

```
- name: rest
```

```
url: http://postgrest:3000
```

```
roads:
```

```
- name: rest-routes
```

```
paths: [/rest/v1]
```

```
strip_path: true
```

```
plugins:
```

```
- name: key-auth
```

```
- name: jwt
```

```
config:
```

```
header_names: [authorization]
```

```
key_claim_name: iss
```

```
claims_to_verify: [exp]
```

```
- name: rate-limiting
```

```
config:
```

```
minute: 180
```

```
hour: 5000
```

Osionos ? Project file

64 / 101

e:rest
url: http://postgrest:3000
roads:
- name: rest-routes
paths: [/rest/v1]
strip_path: true
plugins:
- name: key-auth
- name: jwt
config:
header_names: [authorization]
key_claim_name:iss
claims_to_verify: [exp]
- name: rate-limiting
config:
minute: 180
hour: 5000
Osionos ? Project file
64 / 101

e:rest
url: http://postgrest:3000
roads:
- name: rest-routes
paths: [/rest/v1]
strip_path: true
plugins:
- name: key-auth
- name: jwt
config:
header_names: [authorization]
key_claim_name:iss
claims_to_verify: [exp]
- name: rate-limiting
config:
minute: 180
hour: 5000
Osionos ? Project file
64 / 101

The second scheme is that used by the track-binocle / Prismatic / opposite-osiris profile. He is voluntarily relational: an account has temporary tokens, sessions, activities, GDPR consents and requests. The files that define this model are models/user.sql, models/auth-security-migration.sql and models/gdpr-migration.sql.

The third diagram describes the Osionos part. The browser manipulates pages and workspaces; the backend maintains the lasting correspondence between the Prismatic identity, the private workspace, the pages,

per-user configurations and action events. This part is defined in models/osionos-bridge-migration.sql.

Osionos ? Project file

profile. He is

voluntarily relational: an account has temporary tokens, sessions, activities, GDPR consents and requests. The files that define this model are `models/user.sql`, `models/auth-security-migration.sql` and `models/gdpr-migration.sql`.

The third diagram describes the Osionos part. The browser manipulates pages and workspaces; the backend maintains the lasting correspondence between the Prismatic identity, the private workspace, the pages,

per-user configurations and action events. This part is defined in `models/osionos-bridge-migration.sql`.

Osionos ? Project file

65 / 101

profile. He is

voluntarily relational: an account has temporary tokens, sessions, activities, GDPR consents and requests. The files that define this model are `models/user.sql`, `models/auth-security-migration.sql` and `models/gdpr-migration.sql`.

The third diagram describes the Osionos part. The browser manipulates pages and workspaces; the backend maintains the lasting correspondence between the Prismatic identity, the private workspace, the pages, per-user configurations and action events. This part is defined in `models/osionos-bridge-migration.sql`.

Osionos ? Project file

65 / 101

Finally, the MongoDB plan is more flexible: it does not seek to freeze all forms of documents to be advance. Collections created by schema-service receive a JSON Schema validator, an index on owner_id , and mongo-api or query-router operations systematically inject or filter by owner.

Osionos ? Project file

66 / 101

Finally, the MongoDB plan is more flexible: it does not seek to freeze all forms of documents to be advance. Collections created by schema-service receive a JSON Schema validator, an index on owner_id , and mongo-api or query-router operations systematically inject or filter by owner.

Osionos ? Project file

66 / 101

The second family of scripts is specific to applications: models/user.sql for the user model relational, models/auth-security-migration.sql for authentication auditing, models/gdpr-migration.sql for GDPR consents and requests, and models/osionos-bridge-migration.sql for workspaces/pages Osionos.

```
-- apps/baas/mini-baas-infra/scripts/migrations/postgresql/007_permissions_system.sql
```

```
CREATE TABLE IF NOT EXISTS public.resource_policies (  
  id  
  UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  role_id  
  UUID NOT NULL REFERENCES public.roles(id) ON DELETE CASCADE,  
  resource_type  
  TEXT NOT NULL,  
  resource_name  
  TEXT NOT NULL,  
  actions  
  TEXT[] NOT NULL DEFAULT ARRAY['select'],  
  conditions  
  JSONB DEFAULT '{}::jsonb',  
  effect  
  TEXT NOT NULL DEFAULT 'allow' CHECK (effect IN ('allow', 'deny')),  
  priority  
  INTEGER DEFAULT 0  
);  
  
CREATE OR REPLACE FUNCTION public.has_permission(  
  p_user_id UUID,  
  p_resource_type TEXT,  
  p_resource_name TEXT,  
  p_action TEXT  
  ) RETURNS BOOLEAN AS $fn$  
  DECLARES  
  pol RECORD;  
  found BOOLEAN := false;  
  BEGIN  
  FOR pol IN  
  SELECT rp.effect, rp.conditions  
  FROM public.resource_policies rp  
  JOIN public.user_roles ur ON ur.role_id = rp.role_id  
  WHERE ur.user_id = p_user_id  
  AND (rp.resource_type = p_resource_type OR rp.resource_type = '*')  
  AND (rp.resource_name = p_resource_name OR rp.resource_name = '*')  
  AND p_action = ANY(rp.actions)  
  ORDER BY rp.priority DESC, rp.effect ASC  
  LOOP  
  IF pol.effect = 'deny' THEN  
  RETURN false;  
  END IF;  
  found := true;  
  END LOOP;  
  RETURN found;  
  END;  
  $fn$ LANGUAGE plpgsql STABLE SECURITY DEFINER;
```

Osionos ? Project file

D. EXTRACT FROM THE SQL CREATION SCRIPT AND CONSISTENCY

Data consistency is ensured at several levels, not just by application code.

Foreign keys avoid orphaned data (ON DELETE CASCADE for profiles, tokens, sessions, workspace child pages).

CHECK constraints limit the possible states (visibility , role , engine , effect).

Indexes materialize critical queries (workspace_id , parent_page_id , updated_at , owner_id).

RLS enforces isolation even if an application route goes wrong.

Globally installed realtime triggers allow changes to be propagated without writing a trigger by hand for each future table.

The following snippet shows this last point: the 012_realtime_triggers_all_tables.sql migration installs automatically an AFTER INSERT OR UPDATE OR DELETE trigger on existing and future tables.

-- models/osionos-bridge-migration.sql

```
CREATE TABLE IF NOT EXISTS public.osionos_pages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id UUID NOT NULL REFERENCES public.osionos_workspaces(id) ON DELETE CASCADE,
  parent_page_id UUID REFERENCES public.osionos_pages(id) ON DELETE SET NULL,
  owner_id UUID,
  title TEXT NOT NULL DEFAULT 'Untitled',
  visibility TEXT NOT NULL DEFAULT 'private' CHECK (visibility IN ('private', 'shared', 'public')),
  collaborators JSONB NOT NULL DEFAULT '[]'::jsonb,
  properties JSONB NOT NULL DEFAULT '[]'::jsonb,
  content JSONB NOT NULL DEFAULT '[]'::jsonb,
  archived_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
ALTER TABLE public.osionos_pages ENABLE ROW LEVEL SECURITY;
CREATE POLICY osionos_pages_update_member ON public.osionos_pages
FOR UPDATE TO authenticated USING (
  EXISTS (
    SELECT 1 FROM public.osionos_workspace_members member
    WHERE member.workspace_id = public.osionos_pages.workspace_id
    AND member.user_id = auth.uid()
    AND member.permissions && ARRAY['update', 'admin']::TEXT[]
  )
);
```

- 1.
- 2.
- 3.
- 4.
- 5.

Osionos ? Project file

```
id()  
AND member.permissions && ARRAY['update', 'admin']::TEXT[]  
)  
);  
1.  
2.  
3.  
4.  
5.
```

Osionos ? Project file

69 / 101

id()

AND member.permissions && ARRAY['update', 'admin']::TEXT[]

)

);

1.

2.

3.

4.

5.

Osionos ? Project file

69 / 101

Code snippet, API structure and security

A. TECHNICAL CHOICES, CONTEXT AND LOGIC

NestJS was chosen for services that require clear application logic: DTO validation, dependency injection, REST controllers, guards, Swagger, structured logs and healthchecks. The services share internal libraries (@mini-baas/common , @mini-baas/database) but remain deployable separately using the multi-app Dockerfile.

The bootstrap of a service like query-router shows the common structure: strict validation, filter homogeneous errors, correlation-id, Swagger, clean shutdown.

CREATE OR REPLACE FUNCTION public.realtime_notify()

RETURNS TRIGGER AS \$fn\$

DECLARES

payloadJSON;

BEGIN

payload := json_build_object(

'table',

TG_TABLE_NAME,

'schema',

TG_TABLE_SCHEMA,

'operation', TG_OP,

'data',

CASE WHEN TG_OP = 'DELETE' THEN row_to_json(OLD) ELSE row_to_json(NEW)

END,

'old_data',

CASE WHEN TG_OP = 'UPDATE' THEN row_to_json(OLD) ELSE NULL END

);

PERFORM pg_notify('realtime_events', payload::text);

RETURN COALESCE(NEW, OLD);

END;

\$fn\$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE EVENT TRIGGER realtime_auto_trigger_on_create

ON ddl_command_end

WHEN TAG IN ('CREATE TABLE')

EXECUTE FUNCTION public.realtime_auto_trigger();

Osionos ? Project file

70 / 101

```
xt);  
RETURN COALESCE(NEW, OLD);  
END;  
$fn$ LANGUAGE plpgsql SECURITY DEFINER;  
CREATE EVENT TRIGGER realtime_auto_trigger_on_create  
ON ddl_command_end  
WHEN TAG IN ('CREATE TABLE')  
EXECUTE FUNCTION public.realtime_auto_trigger();  
Osionos ? Project file  
70 / 101
```

```
xt);  
RETURN COALESCE(NEW, OLD);  
END;  
$fn$ LANGUAGE plpgsql SECURITY DEFINER;  
CREATE EVENT TRIGGER realtime_auto_trigger_on_create  
ON ddl_command_end  
WHEN TAG IN ('CREATE TABLE')  
EXECUTE FUNCTION public.realtime_auto_trigger();
```

Osionos ? Project file

70 / 101

The validation is deliberately strict. An unexpected field in a DTO triggers a 400 error instead of being silently accepted.

The user identity is provided by Kong and then read by AuthGuard.

```
// apps/baas/mini-baas-infra/src/apps/query-router/src/main.ts
async function bootstrap() {
  const app = await NestFactory.create(AppModule, { bufferLogs: true });
  app.useLogger(app.get(PinoLogger));
  app.useGlobalPipes(createValidationPipe());
  app.useGlobalFilters(new AllExceptionsFilter());
  app.useGlobalInterceptors(new CorrelationIdInterceptor());
  app.enableShutdownHooks();
  const swaggerConfig = new DocumentBuilder()
    .setTitle('Query Router')
    .setDescription('Universal data plane ? routes queries to registered databases')
    .setVersion('2.0.0')
    .build();
  const config = app.get(ConfigService);
  const port = config.get<number>('PORT', 4001);
  await app.listen(port);
}
// apps/baas/mini-baas-infra/src/libs/common/src/pipes/validation.pipe.ts
export function createValidationPipe(): NestValidationPipe {
  return new NestValidationPipe({
    whitelist: true,
    forbidNonWhitelisted: true,
    transform: true,
    transformOptions: { enableImplicitConversion: true },
  });
}
```

Osionos ? Project file

71 / 101

```
/baas/mini-baas-infra/src/libs/common/src/pipes/validation.pipe.ts
export function createValidationPipe(): NestValidationPipe {
  return new NestValidationPipe({
    whitelist: true,
    forbidNonWhitelisted: true,
    transform: true,
    transformOptions: { enableImplicitConversion: true },
  });
}
```

Osionos ? Project file

71 / 101

/baas/mini-baas-infra/src/libs/common/src/pipes/validation.pipe.ts

```
export function createValidationPipe(): NestValidationPipe {  
  return new NestValidationPipe({  
    whitelist: true,  
    forbidNonWhitelisted: true,  
    transform: true,  
    transformOptions: { enableImplicitConversion: true },  
  });  
}
```

Osionos ? Project file

71 / 101

Sensitive internal calls, for example query-router which asks adapter-registry to decrypt a connection, do not go through a classic user JWT. They use a service token and an explicit X-Tenant-Id. The called service then reconstitutes a service_role context limited to now asked.

B. EXTRACT 1: OWNERSHIP CONTROL AND CRITICAL AUTHORIZATION RULE

Ownership control is deliberately doubled: PostgreSQL does it with RLS, and MongoDB does it with an owner_id filter injected into queries. This dual model is necessary because PostgreSQL knows how to apply a policy at the row level, while MongoDB requires doing so in the row level application layer.

```
// apps/baas/mini-baas-infra/src/libs/common/src/guards/auth.guard.ts
```

```
@Injectable()
export class AuthGuard implements CanActivate {
  canActivate(context: ExecutionContext): boolean {
    const req = context.switchToHttp().getRequest<Request>();
    const userId = req.headers['x-user-id'] as string | undefined;
    const email = req.headers['x-user-email'] as string | undefined;
    const role = req.headers['x-user-role'] as string | undefined;
    if (!userId) {
      throw new UnauthorizedException('Missing authentication ? X-User-Id header required');
    }
    req.user = {
      id: userId,
      email: email ?? '',
      role: role ?? 'authenticated',
    } satisfies UserContext;
    return true;
  }
}
```

```
// apps/baas/mini-baas-infra/src/libs/common/src/guards/service-token.guard.ts
```

```
if (serviceToken && expectedToken && serviceToken === expectedToken) {
  const tenantId = req.headers['x-tenant-id'] as string | undefined;
  if (!tenantId) {
    throw new UnauthorizedException('Service token requires X-Tenant-Id header');
  }
  req.user = {
    id: tenantId,
    email: 'service@internal',
    role: 'service_role',
  } satisfies UserContext;
  return true;
}
```

Osionos ? Project file


```
id: tenantId,  
email: 'service@internal',  
role: 'service_role',  
} satisfies UserContext;  
return true;  
}
```

Osionos ? Project file

72 / 101

```
id: tenantId,  
email: 'service@internal',  
role: 'service_role',  
} satisfies UserContext;  
return true;  
}
```

Osionos ? Project file

72 / 101

On the PostgreSQL side, tenant queries go through `tenantQuery()`. The method opens a transaction, sets the local variable `app.current_user_id`, executes the query, then commit or rollback. The SQL policies can then compare `owner_id` or `tenant_id` to this value.

On the MongoDB side, the service removes the prohibited fields (`_id`, `owner_id`) provided by the client, injects its

own `owner_id`, and adds this owner in all `find`, `patch` and `delete`.

```
// apps/baas/mini-baas-infra/src/libs/database/src/postgres/postgres.service.ts
```

```
async tenantQuery<T extends QueryResultRow = Record<string, unknown>>(  
  userId: string,  
  text: string,  
  params?: unknown[],  
) : Promise<T[]> {  
  const client: PoolClient = await this.tenantPool.connect();  
  try {  
    await client.query('BEGIN');  
    await client.query(`SET LOCAL app.current_user_id = $1`, [userId]);  
    const result = await client.query<T>(text, params);  
    await client.query('COMMIT');  
    return result.rows;  
  } catch(err) {  
    await client.query('ROLLBACK');  
    throw err;  
  } finally {  
    client.release();  
  }  
}
```

Osionos ? Project file

73 / 101

```
his.tenantPool.connect();
try {
  await client.query('BEGIN');
  await client.query(`SET LOCAL app.current_user_id = $1`, [userId]);
  const result = await client.query<T>(text, params);
  await client.query('COMMIT');
  return result.rows;
} catch(err) {
  await client.query('ROLLBACK');
  throw err;
} finally {
  client.release();
}
}
```

Osionos ? Project file

73 / 101

```
his.tenantPool.connect();
try {
  await client.query('BEGIN');
  await client.query(`SET LOCAL app.current_user_id = $1`, [userId]);
  const result = await client.query<T>(text, params);
  await client.query('COMMIT');
  return result.rows;
} catch(err) {
  await client.query('ROLLBACK');
  throw err;
} finally {
  client.release();
}
}
```

Osionos ? Project file

73 / 101

The same principle exists in Osionos, but at the workspace/page scale. Before reading or modifying a page, the bridge checks both the application token and workspace membership in PostgREST.

```
// apps/baas/mini-baas-infra/src/apps/mongo-api/src/collections/collections.service.ts
async create(collectionName: string, userId: string, data: Record<string, unknown>) {
  const { _id: _, owner_id: __, ...clean } = data;
  const doc = {
    ...clean,
    owner_id: userId,
    created_at: new Date(),
    updated_at: new Date(),
  };
  const col = this.getCollection(collectionName);
  const result = await col.insertOne(doc);
  return this.normalizeDoc({ _id: result.insertedId, ...doc });
}

async patch(collectionName: string, userId: string, docId: string, patch:
Record<string, unknown>) {
  const { _id: _, owner_id: __, ...clean } = patch;
  const result = await col.findOneAndUpdate(
    { _id: new ObjectId(docId), owner_id: userId },
    { $set: { ...clean, updated_at: new Date() } },
    { returnDocument: 'after' },
  );
}

// apps/osionos/app/scripts/bridge-api.mjs
export async function requireWorkspaceAccess(request, workspaceId, permission, config, fetchImpl = fetch) {
  const normalizedWorkspaceId = requireUuid(workspaceId, 'workspaceId');
  const authContext = verifyAppSessionToken(bearerToken(request), config);
  if (!authContext.workspaceIds.includes(normalizedWorkspaceId)) {
    throw Object.assign(new Error('App session is not scoped to this workspace.'), { status: 403 });
  }
  const query = postgrestQuery({
    workspace_id: `eq.${normalizedWorkspaceId}`,
    user_id: `eq.${authContext.userId}`,
    select: 'role,permissions',
    limit: '1',
  });
  const rows = await baasRest(config, fetchImpl, `osionos_workspace_members?${query}`);
  const member = Array.isArray(rows) ? rows[0] : null;
  if (!memberHasPermission(member, normalizePermission(permission))) {
    throw Object.assign(new Error('Workspace permission denied.'), { status: 403 });
  }
}
```

Osionos ? Project file

```
Error('Workspace permission denied.'). { status: 403 };  
}  
}
```

Osionos ? Project file

74 / 101

```
Error('Workspace permission denied.'). { status: 403 });  
}  
}
```

Osionos ? Project file

74 / 101

C. EXTRACT 2: CRITICAL BUSINESS ACTION, CONNECT A BASIS AND USE IT

A critical backend action is the connection of an external database. This action affects several risks: connection secret, ownership, base engine, schema validation, then execution of query. This is why it crosses several services.

The REST controller of the query-router exposes only two families of actions: execute on a table/collection or list the available tables/collections.

The query-router never directly knows the stored connection secrets. He asks them to registry via internal HTTP, with a service token.

```
// apps/baas/mini-baas-infra/src/apps/query-router/src/query/query.controller.ts
```

```
@ApiTags('query')
@Controller('query')
@UseGuards(AuthGuard)
export class QueryController {
  @Post('/:dbId/tables/:table')
  async execute(
    @CurrentUser() user: UserContext,
    @Param('dbId', ParseUUIDPipe) dbId: string,
    @Param('table') table: string,
    @Body() dto: ExecuteQueryDto,
  ) {
    return this.service.executeQuery(dbId, table, user.id, dto);
  }
  @Get('/:dbId/tables')
  async listTables(@CurrentUser() user: UserContext, @Param('dbId', ParseUUIDPipe)
    dbId: string) {
    return this.service.listTables(dbId, user.id);
  }
}
```

Osionos ? Project file

```
aram('table') table: string,  
@Body() dto: ExecuteQueryDto,  
) {  
    return this.service.executeQuery(dbId, table, user.id, dto);  
}  
@Get('/:dbId/tables')  
async listTables(@CurrentUser() user: UserContext, @Param('dbId', ParseUUIDPipe)  
dbId: string) {  
    return this.service.listTables(dbId, user.id);  
}  
}
```

Osionos ? Project file

75 / 101

```
aram('table') table: string,  
@Body() dto: ExecuteQueryDto,  
) {  
    return this.service.executeQuery(dbId, table, user.id, dto);  
}  
@Get('/:dbId/tables')  
async listTables(@CurrentUser() user: UserContext, @Param('dbId', ParseUUIDPipe)  
dbId: string) {  
    return this.service.listTables(dbId, user.id);  
}  
}  
Osionos ? Project file  
75 / 101
```

The registry encrypts at the time of recording, then decrypts only for authorized calls.

For PostgreSQL, query-router validates table/column names, sets values, injects owner_id to the insert and sets the RLS context before executing.

For MongoDB, the engine applies the proprietary filter, limits results, and removes builds dangerous like \$where .

```
// apps/baas/mini-baas-infra/src/apps/query-router/src/query/query.service.ts
private async fetchConnection(dbId: string, userId: string): Promise<AdapterResponse> {
  const url = `${this.registryUrl}/databases/${dbId}/connect`;
  const { data } = await firstValueFrom(
    this.http.get<AdapterResponse>(url, {
      headers: {
        'X-Service-Token': this.serviceToken,
        'X-Tenant-Id': userId,
      },
    }),
  );
  return data;
}

// apps/baas/mini-baas-infra/src/apps/adapter-registry/src/crypto/crypto.service.ts
encrypt(plaintext: string): EncryptedPayload {
  const salt = randomBytes(SALT_LENGTH);
  const key = scryptSync(this.masterKey, salt, KEY_LENGTH);
  const iv = randomBytes(IV_LENGTH);
  const cipher = createCipheriv(ALGORITHM, key, iv);
  const encrypted = Buffer.concat([cipher.update(plaintext, 'utf8'), cipher.final()]);
  const tag = cipher.getAuthTag();
  return { encrypted, iv, tag, salt };
}

// apps/baas/mini-baas-infra/src/apps/query-router/src/engine/postgresql.engine.ts
const TABLE_REGEX = /^[a-zA-Z_]\w{0,63}$/;
const COLUMN_REGEX = /^[a-zA-Z_]\w*$/;
if (opts.userId) {
  await client.query('BEGIN');
  await client.query(`SET LOCAL app.current_user_id = $1`, [opts.userId]);
}
const enriched = { ...data };
if (userId && !enriched['owner_id']) {
  enriched['owner_id'] = userId;
}
```

Osionos ? Project file

76 / 101

```
.current_user_id = $1`, [opts.userId]);  
}  
const enriched = { ...data };  
if (userId && !enriched['owner_id']) {  
  enriched['owner_id'] = userId;  
}
```

Osionos ? Project file

76 / 101

```
.current_user_id = $1`, [opts.userId]);  
}  
const enriched = { ...data };  
if (userId && !enriched['owner_id']) {  
  enriched['owner_id'] = userId;  
}
```

Osionos ? Project file

76 / 101

D. EXTRACT 3: OSIONOS BUSINESS ACTION, CREATE AND MODIFY A PAGE

Osionos has a lighter backend, written in native Node in bridge-api.mjs. Its role is to receive assertion signed from Prismatic, create a short application session, then serve REST routes to the pages. This is where we see the real connection between the rich front and BaaS.

The first barrier is HMAC: Prismatic signs the payload with a shared secret, the bridge verifies the timestamp, signature and jti to avoid replay.

```
// apps/baas/mini-baas-infra/src/apps/query-router/src/engines/mongodb.engine.ts
private applyOwnerFilter(filter: Record<string, unknown>, userId?: string): Record<string, unknown> {
  if (userId) {
    filter['owner_id'] = userId;
  }
  return filter;
}

private async find(col: Collection, opts: MongoExecuteOptions):
Promise<MongoQueryResult> {
  const filter = this.applyOwnerFilter(this.cloneFilter(opts.filter), opts.userId);
  delete filter['$where'];
  const limit = Math.min(opts.limit ?? 100, 100);
  let cursor = col.find(filter).skip(opts.offset ?? 0).limit(limit);
  const sort = this.buildSort(opts.sort);
  if (sort) {
    cursor = cursor.sort(sort);
  }
  const docs = await cursor.toArray();
  return {
    rows: docs.map((d) => this.normalizeDoc(d as Record<string, unknown>)),
    rowCount: docs.length,
  };
}
```

Osionos ? Project file

77 / 101

```
r).skip(opts.offset ?? 0).limit(limit);
const sort = this.buildSort(opts.sort);
if (sort) {
  cursor = cursor.sort(sort);
}
const docs = await cursor.toArray();
return {
  rows: docs.map((d) => this.normalizeDoc(d as Record<string, unknown>)),
  rowCount: docs.length,
};
}
```

Osionos ? Project file

77 / 101


```
r).skip(opts.offset ?? 0).limit(limit);
const sort = this.buildSort(opts.sort);
if (sort) {
  cursor = cursor.sort(sort);
}
const docs = await cursor.toArray();
return {
  rows: docs.map((d) => this.normalizeDoc(d as Record<string, unknown>)),
  rowCount: docs.length,
};
}
```

Osionos ? Project file

77 / 101

Then, the page routes remain REST: GET /api/pages, POST /api/pages, PATCH /api/pages/:id , DELETE /api/pages/:id . Each writing goes through requireWorkspaceAccess() .

```
// apps/osionos/app/scripts/bridge-api.mjs
export function verifyBridgeRequest({ headers, payload, secret, now = Date.now(), replay
Store = new Map() }) {
  const timestampHeader = headers['x-prismatica-bridge-timestamp'];
  const signatureHeader = headers['x-prismatica-bridge-signature'];
  const timestamp = Number(timestampHeader);
  if (!Number.isFinite(timestamp) || Math.abs(now - timestamp) > DEFAULT_TIMESTAMP_SKEW_
MS) {
    throw Object.assign(new Error('Bridge assertion timestamp is outside the allowed
window.'), { status: 401 });
  }
  const normalizedPayload = validateBridgePayload(payload);
  const expected = bridgeSignature(secret, String(timestampHeader), normalizedPayload);
  if (typeof signatureHeader !== 'string' || !safeCompareHex(expected,
signatureHeader)) {
    throw Object.assign(new Error('Bridge signature is invalid.'), { status: 401 });
  }
  if (replayStore.has(normalizedPayload.jti)) {
    throw Object.assign(new Error('Bridge assertion replay rejected.'), { status: 409 })
  }
  ;
  replayStore.set(normalizedPayload.jti, { expiresAt: now + DEFAULT_TIMESTAMP_SKEW_MS })
  ;
  return normalizedPayload;
}

// apps/osionos/app/scripts/bridge-api.mjs
async function handlePageUpdate(url, request, response, config, fetchImpl) {
  const pageId = pageIdFromPath(url.pathname);
  if (!pageId) return false;
  const existing = await fetchPageRow(pageId, config, fetchImpl);
  if (!existing) throw Object.assign(new Error('Page not found.'), { status: 404 });
  await requireWorkspaceAccess(request, existing.workspace_id, 'update', config, fetchIm
pl);
  const payload = await readJson(request, PAGE_JSON_BODY_LIMIT_BYTES);
  const updateRow = pageUpdateRowFromPayload(payload);
  const rows = await baasRest(config, fetchImpl, `osionos_pages?id=eq.${pageId}`, {
    method: 'PATCH',
    body: updateRow,
    prefer: 'return=representation',
  });
  json(response, 200, pageRowToEntry(Array.isArray(rows) ? rows[0]: rows), config);
  return true;
}
```

Osionos ? Project file

```
rows) ? rows[0]: rows), config);  
return true;  
}
```

Osionos ? Project file

78 / 101

```
rows) ? rows[0]: rows), config);  
return true;  
}
```

Osionos ? Project file

78 / 101

Data Access Component (DAO) Code Snippets

A. TECHNICAL CHOICES, CONTEXT AND LOGIC

In this project, I did not create a layer of frozen repositories like in a CRUD back-end classic. The need was broader: we had to talk to PostgreSQL, MongoDB, MinIO, PostgREST and dynamically recorded external databases. The role of DAO is therefore carried out by access services to data and engines:

PostgresService: admin pool + tenant pool with RLS context.

MongoService: shared MongoDB client, pool, healthcheck.

DatabasesService: database registry and connection encryption.

QueryService: orchestration between user, adapter-registry and engine.

PostgresqlEngine / MongoddbEngine: concrete execution of operations.

SchemasService: creation of tables/collections from a unified schema.

This choice also explains why we did not choose Prisma as the main ORM. Prisma is excellent when the relational model is stable, known in advance and mostly PostgreSQL/MySQL. Here, part of the product is based on user-created diagrams, external bases recorded at runtime, a PostgreSQL + MongoDB execution, and a strong dependence on RLS and SQL session variables (SET LOCAL app.current_user_id). A statically generated client would have been less suitable. The cost of this choice is that we lose part of the type-safe comfort of an ORM; we compensated by strict DTOs, identifier name regex, parameterized queries, policies SQL and targeted tests.

It is also necessary to be precise: the registry today accepts the engines postgresql, mongodb, mysql, redis , sqlite , but the execution checked in query-router is implemented for PostgreSQL and MongoDB. The other engines are prepared on the model side, not yet full execution paths.

B. EXTRACT 1: OWNER-SCOPED DATA RECOVERY

MongoDB document retrieval does not depend on a filter sent from the front. Even if the customer sends a filter, the service adds owner_id = userId and removes fields that should not be controlled by the customer.

?

?

?

?

?

?

Osionos ? Project file

DB document retrieval does not depend on a filter sent from the front. Even if the customer sends a filter, the service adds owner_id = userId and removes fields that should not be controlled by the customer.

?

?

?

?

?

?

Osionos ? Project file

79 / 101

DB document retrieval does not depend on a filter sent from the front. Even if the customer sends a filter, the service adds owner_id = userId and removes fields that should not be controlled by the customer.

?

?

?

?

?

?

Osionos ? Project file

79 / 101

C. EXTRACT 2: CREATION OF A DIAGRAM USABLE BY SEVERAL ENGINES

schema-service is a good example of a backend business service: it does not just do a CREATE TABLE . It checks that the requested engine corresponds to the registered base, creates the structure engine side, then writes a trace to schema_registry .

```
// apps/baas/mini-baas-infra/src/apps/mongo-api/src/collections/collections.service.ts
```

```
async findAll(
  collectionName: string,
  userId: string,
  opts: { limit: number; offset: number; sort?: string; filter?: string },
) {
  const col = this.getCollection(collectionName);
  const query: Record<string, unknown> = { owner_id: userId };
  if (opts.filter) {
    const parsed = JSON.parse(opts.filter) as Record<string, unknown>;
    delete parsed['owner_id'];
    delete parsed['_id'];
    Object.assign(query, parsed);
  }
  let sort: Sort = { created_at: -1 };
  if (opts.sort) {
    const [field, dir] = opts.sort.split(':');
    if (field && dir) {
      sort = { [field]: dir.toLowerCase() === 'asc' ? 1: -1 };
    }
  }
  const [data, total] = await Promise.all([
    col.find(query).sort(sort).skip(opts.offset).limit(opts.limit).toArray(),
    col.countDocuments(query),
  ]);
  return { data: data.map((d) => this.normalizeDoc(d as Record<string, unknown>)),
    meta: { total, limit: opts.limit, offset: opts.offset } };
}
```

Osionos ? Project file


```
d(query).sort(sort).skip(opts.offset).limit(opts.limit).toArray(),
col.countDocuments(query),
]);
return { data: data.map((d) => this.normalizeDoc(d as Record<string, unknown>)),
meta: { total, limit: opts.limit, offset: opts.offset } };
}
```

Osionos ? Project file

80 / 101

```
d(query).sort(sort).skip(opts.offset).limit(opts.limit).toArray(),
col.countDocuments(query),
]);
return { data: data.map((d) => this.normalizeDoc(d as Record<string, unknown>)),
meta: { total, limit: opts.limit, offset: opts.offset } };
}
```

Osionos ? Project file

80 / 101

The PostgreSQL part automatically adds `id` , `owner_id` , `created_at` , `updated_at` , then installs an `owner_isolation` policy if `enable_rls` is active.

```
// apps/baas/mini-baas-infra/src/apps/schema-service/src/schemas/schemas.service.ts
async create(userId: string, dto: CreateSchemaDto) {
  const { engine, connection_string } = await this.fetchConnection(dto.database_id, user
  ID);
  if (engine !== dto.engine) {
    throw new BadRequestException(
      `Engine mismatch ? database is ${engine} but schema spec says ${dto.engine}`,
    );
  }
  if (engine === 'postgresql') {
    const result = await this.pgEngine.createTable(
      connection_string,
      dto.name,
      dto.columns,
      dto.enable_rls !== false,
    );
    await this.pg.adminQuery(
      `INSERT INTO schema_registry (database_id, name, engine, columns, enable_rls,
      created_by)
      VALUES ($1, $2, $3, $4::jsonb, $5, $6)
      ON CONFLICT (database_id, name) DO UPDATE SET columns = $4::jsonb, enable_rls =
      $5`,
      [dto.database_id, dto.name, engine, JSON.stringify(dto.columns), dto.enable_rls !
      == false, userId],
    );
    return result;
  }
}
```

Osionos ? Project file

81 / 101

```
d_by)
    VALUES ($1, $2, $3, $4::jsonb, $5, $6)
    ON CONFLICT (database_id, name) DO UPDATE SET columns = $4::jsonb, enable_rls =
$5`,
[dto.database_id, dto.name, engine, JSON.stringify(dto.columns), dto.enable_rls !
== false, userId],
);
return result;
}
}
```

Osionos ? Project file

81 / 101

```
d_by)
VALUES ($1, $2, $3, $4::jsonb, $5, $6)
ON CONFLICT (database_id, name) DO UPDATE SET columns = $4::jsonb, enable_rls =
$5`,
[dto.database_id, dto.name, engine, JSON.stringify(dto.columns), dto.enable_rls !
== false, userId],
);
return result;
}
}
```

Osionos ? Project file

81 / 101

The MongoDB part creates or updates a JSON Schema validator and an index useful for owner-queries scoped.

D. DEPLOYMENT PREPARATION, ORCHESTRATION AND RECOVERY

The deployment is prepared with Docker Compose and a multi-stage Dockerfile. The idea is not to build a different image by hand for each NestJS service: the same Dockerfile receives ARG APP , compiles only the requested application, removes development dependencies, then runs the service with a non-root user.

```
// apps/baas/mini-baas-infra/src/apps/schema-service/src/engines/postgres-  
schema.engine.ts
```

```
const colDefs: string[] = [  
  `id UUID PRIMARY KEY DEFAULT gen_random_uuid()`,  
  `owner_id UUID NOT NULL`,  
  `created_at TIMESTAMPTZ DEFAULT now()`,  
  `updated_at TIMESTAMPTZ DEFAULT now()`,  
];  
await client.query(`ALTER TABLE public."${tableName}" ENABLE ROW LEVEL SECURITY`);  
await client.query(  
  `DO $$ BEGIN  
    IF NOT EXISTS (SELECT 1 FROM pg_policies WHERE tablename = '${tableName}' AND  
policyname = 'owner_isolation') THEN  
      CREATE POLICY owner_isolation ON public."${tableName}" FOR ALL  
        USING (owner_id::text = current_user_id())  
        WITH CHECK (owner_id::text = current_user_id());  
    END IF;  
  END $$`,  
);
```

```
// apps/baas/mini-baas-infra/src/apps/schema-service/src/engines/mongo-schema.engine.ts
```

```
const properties: Record<string, unknown> = {  
  owner_id: { bsonType: 'string' },  
  created_at: { bsonType: 'date' },  
  updated_at: { bsonType: 'date' },  
};  
if (existing.length) {  
  await db.command({ collMod: collectionName, validator, validationLevel: 'strict' });  
} else {  
  await db.createCollection(collectionName, { validator });  
  await db.collection(collectionName).createIndex({ owner_id: 1, created_at: -1 });  
}
```

Osionos ? Project file

```
t' });  
} else {  
  await db.createCollection(collectionName, { validator });  
  await db.collection(collectionName).createIndex({ owner_id: 1, created_at: -1 });  
}
```

Osionos ? Project file

82 / 101

```
t' });  
} else {  
  await db.createCollection(collectionName, { validator });  
  await db.collection(collectionName).createIndex({ owner_id: 1, created_at: -1 });  
}
```

Osionos ? Project file

82 / 101

Compose orchestrates dependencies with depends_on , healthcheck , restart: unless-stopped , persistent volumes, CPU/memory limits and profiles (data-plane, control-plane, adapter-plane, storage, observability). Example with query-router: it only starts if adapter-registry and permission-engine are healthy, and it talks to other services by Docker DNS.

```
# apps/baas/mini-baas-infra/src/Dockerfile
FROM public.ecr.aws/docker/library/node:${NODE_VERSION}-alpine AS deps
WORKDIR /app
COPY --link package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm,sharing=locked \
npm ci --ignore-scripts --prefer-offline --no-audit --no-fund
FROM deps AS build
ARG APP
COPY --link tsconfig.json tsconfig.build.json nest-cli.json ./
COPY --link libs/ ./libs/
COPY --link apps/${APP}/ ./apps/${APP}/
RUN npx nest build ${APP}
FROM public.ecr.aws/docker/library/node:${NODE_VERSION}-alpine AS runtime
ARG APP
ENV NODE_ENV=production APP_NAME=${APP}
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER press
CMD ["sh", "-c", "node dist/apps/${APP_NAME}/apps/${APP_NAME}/src/main.js"]
# apps/baas/mini-baas-infra/docker-compose.yml
query-router:
build:
context: ./src
dockerfile: Dockerfile
args:
APP: query router
environment:
PORT: 4001
ADAPTER_REGISTRY_URL: http://adapter-registry:3020
PERMISSION_ENGINE_URL: http://permission-engine:3050
QUERY_ROUTER_REDIS_URL: redis://redis:6379
ADAPTER_REGISTRY_SERVICE_TOKEN: ${ADAPTER_REGISTRY_SERVICE_TOKEN}
depends_on:
adapter-registry:
condition: service_healthy
permission-engine:
condition: service_healthy
say again:
condition: service_started
networks:
- mini-baas
restart: unless-stopped
healthcheck:
test: ["CMD-SHELL", "wget -qO- http://localhost:4001/health/live || exit 1"]
Osionos ? Project file
83 / 101
```

lth/live || exit 1"]

Osionos ? Project file

83 / 101

lth/live || exit 1"]
Osionos ? Project file
83 / 101

The production overlay docker-compose.prod.yml removes direct ports from the bases (postgres, mongo, gotrue, postgres, redis) and maintains access via the intended services. This limits the surface exposure: in production, the database is not supposed to be called directly from the outside. If an application server goes down, Compose can restart it using healthchecks and policies restart. If mongo-api goes down, the documents don't disappear: they are in the mongo-api volume. data. If query-router crashes, it loses its memory caches, but the data remains in PostgreSQL, MongoDB or the external database. If postgres restarts, the postgres-data volume retains the data. If realtime-agnostic restarts, the next changes start from the base; the basis remains the source of truth.

However, we must be honest: this Compose configuration is robust for a local environment, demo or a small deployment, but it is not yet multi-node high availability.

There is no automatic multi-replica PostgreSQL failover in this file. For a production critical, it would be necessary to add a planned backup strategy, external storage, replicas, alerting supervision and tested restoration procedures.

Backup/restore scripts already exist for PostgreSQL and MongoDB. They show the direction operational: pg_dump in custom format for PostgreSQL, mongodump in archive for MongoDB, then explicit restoration.

```
# apps/baas/mini-baas-infra/docker-compose.prod.yml
```

```
postgres:
```

```
ports: []
```

```
deploy:
```

```
resources:
```

```
limits:
```

```
memory: 512m
```

```
cpu: '0.50'
```

```
restart_policy:
```

```
condition: on-failure
```

```
delay: 5s
```

```
max_attempts: 3
```

```
mongo:
```

```
ports: []
```

```
# apps/baas/mini-baas-infra/docker/services/postgres/tools/backup.sh
```

```
BACKUP_FILE="backup_$(date +%Y%m%d).dump"
```

```
docker compose exec postgres pg_dump -U postgres -Fc > "${BACKUP_FILE}"
```

```
# apps/baas/mini-baas-infra/docker/services/postgres/tools/restore.sh
```

```
docker compose exec -T postgres pg_restore -U postgres -d postgres < "${BACKUP_FILE}"
```

```
# apps/baas/mini-baas-infra/docker/services/mongo/tools/backup.sh
```

```
BACKUP_FILE="mongo_backup_$(date +%Y%m%d).archive"
```

```
docker compose exec mongo mongodump --archive > "${BACKUP_FILE}"
```

Osionos ? Project file

vices/mongo/tools/backup.sh

BACKUP_FILE="mongo_backup_\$(date +%Y%m%d).archive"

docker compose exec mongo mongodump --archive > "\${BACKUP_FILE}"

Osionos ? Project file

84 / 101

vices/mongo/tools/backup.sh

```
BACKUP_FILE="mongo_backup_$(date +%Y%m%d).archive"
```

```
docker compose exec mongo mongodump --archive > "${BACKUP_FILE}"
```

Osionos ? Project file

84 / 101

The secrets are not integrated into the images. The control-plane profile contains Vault, and scripts environment retrieve or generate the necessary values without writing them clearly in the code source. The ensure-osionos-runtime-secrets.mjs script generates, for example, the secrets of the Osionos bridge

locally, while vault-env.mjs centralizes the variable families expected for the services.

In summary, the back-end was designed as a platform: REST on the front, specialized services behind Kong, persistent databases, RLS and ownership as close as possible to the data, encrypted secrets, deployment

reproducible, and an assumed limit between what is already robust in Compose and what would require a complete high availability architecture.

CHAPTER 5. Application security elements

Security is probably the part of the project where I learned the most to say ?I don?t know, we?ll check?. Code that works is easy ? secure code is verified.

The security architecture is based on two central services: GoTrue (authentication, bcrypt hashing, emission of JWTs) and Kong (API gateway, verification of JWTs, CORS control, injection of claims in internal headers). The planned public flow goes through WAF then Kong; in development, certain ports premises remain voluntarily exposed for debugging, and the production overlay removes direct access to the basics. This chapter details how these two services come together with the application layers.

Authentication and role management

The authentication service ? GoTrue

We did not rewrite our own auth server. We chose GoTrue v2.188.1, the open-source service that Supabase uses in production. The logic: an auth service is something where a subtle error costs expensive (timing attacks, stolen sessions, etc.), so you might as well take a battle-tested project rather than act smart.

Configuration in docker-compose.yml:199-240:

JWT signed in HS256 (symmetric key shared between GoTrue and Kong)

Expiration of 3600 seconds (1 hour) for access tokens

The JWT_SECRET is provided to GoTrue by the runtime environment, generated or retrieved via the workflow Vault/Makefile ? not in the clear in the code, not committed. The script that describes these variable families is vault-env.mjs.

Password hashing ? bcrypt

GoTrue uses bcrypt by default to hash passwords before inserting them into auth.users. It is the industry standard, resistant to brute force thanks to the adaptive cost factor. We didn?t touch that ? this is exactly why we took GoTrue rather than coding our own hashPassword() with incorrectly configured crypto.pbkdf2().

The login flow

Concretely, when a user connects:

The React front sends email + password to /api/auth/login (useAuth.ts:221-226)

The intermediate gateway (auth-gateway.mjs) validates the fields, then calls the BaaS SDK (auth-gateway.mjs:859-886)

?

?

?

1.

2.

Osionos ? Project file

lls the BaaS SDK (auth-
gateway.mjs:859-886)

?

?

?

1.

2.

Osionos ? Project file

85 / 101

Is the BaaS SDK (auth-
gateway.mjs:859-886)

?

?

?

1.

2.

Osionos ? Project file

85 / 101

The SDK does a POST to GoTrue: /auth/v1/token?grant_type=password (sdk/src/domains/auth.ts: 41-50)

GoTrue checks the bcrypt, signs a JWT, returns access_token + refresh_token

The refresh_token is stored in cookie HttpOnly + Secure + SameSite=Lax (auth-gateway.mjs: 141-147) ? this is important to resist theft by XSS

Checking the JWT ? Kong in the middle

Rather than each microservice verifying the JWT, it is Kong (the API gateway) which does it once and for all: Kong intercepts the request, validates the signature, checks exp, then decodes the claims and re-injects them into

headers to microservices (kong.yml:69-101):

X-User-Id ? claim sub

X-User-Email ? claim email

X-User-Role ? claim role

Microservices trust these headers in the normal flow because they are behind Kong on the Docker network. Concretely: an attacker cannot send X-User-Id: 1 directly to mongo-api from the host, because that port is not mapped. For services and bases that expose a port local under development, the production overlay reduces this surface and access control must remain worn by Kong, the guards and the base.

Role management ? RBAC + ABAC

The permissions system goes further than a simple RBAC. We have an ABAC (Attribute-Based Access Control) which overlaps with the roles.

The roles are defined in base in 007_permissions_system.sql:71-77:

admin ? complete platform

user ? standard user (CRUD only on what it owns)

guest ? read-only

moderator ? content moderation

service_role ? internal service-to-service identity

On the NestJS side, we have a RolesGuard which is applied after the AuthGuard. Actual code (roles.guard.ts:35-60):

3.

4.

5.

apps/baas/mini-baas-infra/docker/services/kong/conf/kong.yml:15-24

consumers:

- username: authenticated

jwt_secrets:

- algorithm: HS256

key: track-binocle-iss

secret: \${JWT_SECRET}

?

?

?

?

?

?

?

?

Osionos ? Project file

T_SECRET}

?

?

?

?

?

?

?

?

Osionos ? Project file

86 / 101

T_SECRET}

?

?

?

?

?

?

?

?

Osionos ? Project file

Concrete use (permissions.controller.ts:39-66):

The ABAC part ? evaluation by attribute, not by raw role

This is where it becomes interesting to explain. A classic RBAC says ?you are a member, you can can_edit ?. An ABAC says ?for this specific resource, based on who you are and what attributes apply, you have this level of permission?. The difference is that we can give can_view to a user specific on a page, even if its workspace role would normally give it can_edit .

The access rule is stored in the MongoDB AccessRule model (accessRule.model.ts). Every rule has a target which can be:

And the explicit: boolean flag which determines whether the rule overwrites (true) or inherits (false) the rules more

general.

@Injectable()

```
export class RolesGuard implements CanActivate {
  constructor(private readonly reflector: Reflector) {}
  canActivate(context: ExecutionContext): boolean {
    const requiredRoles = this.reflector.getAllAndOverride<string[]>(ROLES_KEY, [
      context.getHandler(),
      context.getClass(),
    ]);
    if (!requiredRoles?.length) return true;
    const req = context.switchToHttp().getRequest<Request>();
    const userRole = req.user?.role;
    if (!userRole || !requiredRoles.includes(userRole)) {
      throw new ForbiddenException(
        `Insufficient permissions ? requires one of: ${requiredRoles.join(', ')}`,
      );
    }
    return true;
  }
}
```

@Delete('roles/:userId/:roleName')

@UseGuards(RolesGuard)

@Roles('admin', 'service_role')

async revoke(...) { ... }

// target.type = 'user' ? rule on a specific person

// target.type = 'role' ? rule on a workspace role

// target.type = 'workspace' ? default rule for the entire workspace

// target.type = 'public' ? unauthenticated access

target: {

type: 'user' | 'role' | 'workspace' | 'public',

userId?: ObjectId,

// if type = 'user'

role?:

thong,

// if type = 'role'

}

Osionos ? Project file

87 / 101

```
pe = 'user'
role?:
thong,
// if type = 'role'
}
Osionos ? Project file
87 / 101
```

```
pe = 'user'
```

```
role?:
```

```
thong,
```

```
// if type = 'role'
```

```
}
```

Osionos ? Project file

87 / 101

The resolution cascade in engine.ts:86-134:

Then in resolver.ts:45-63, conflict resolution:

Concrete example: a workspace gives can_edit to members (inherited rule, resourceType: workspace). We can then set a rule explicit: true, can_view, target: {type:

'user', userId: X} on a specific page. Result: this user

read-only page. That's the attribute ? the identity and the target resource determine the right, not the only one role.

JSONB conditions on the SQL side add a third attribute level: the policy can contain

{ "owner_only": true } , which means that the rule only applies if the user is the owner of the resource. Seed in migration (007_permissions_system.sql:234-258):

// All applicable rules: global workspace ? specific page

```
const rules = await AccessRuleModel.find({
```

```
workspaceId,
```

```
$and: [
```

```
{ $or: [
```

```
{ resourceId: null, resourceType: 'workspace' }, // default workspace
```

```
{ resourceId },
```

```
// rule on this resource
```

```
]],
```

```
{ $or: [
```

```
{ 'target.type': 'workspace' },
```

```
// global rule
```

```
{ 'target.type': 'role',
```

```
'target.role': member.role }, // by role
```

```
{ 'target.type': 'user',
```

```
'target.userId': userId },
```

```
// by specific user
```

```
{ 'target.type': 'public' },
```

```
]],
```

```
],
```

```
)).sort({ resourceType: 1 }) // workspace < page < database < block
```

```
export function resolvePermission(
```

```
rules: Array<{ permission: PermissionLevel; explicit: boolean }>,
```

```
): PermissionLevel {
```

```
let effective: PermissionLevel = 'no_access';
```

```
for (const rule of rules) {
```

```
if (rule.explicit) {
```

```
effective = rule.permission;
```

```
// explicit ? overwrite everything
```

```
} else {
```

```
effective = maxPermission(effective, rule.permission); // inherited ? takes most
```

```
top
```

```
}
```

```
}
```

```
return effective;
```

```
}
```

Osionos ? Project file

88 / 101

rited ? takes most

top

}

}

return effective;

}

Osionos ? Project file

88 / 101

rited ? takes most

top

}

}

return effective;

}

Osionos ? Project file

88 / 101

The SQL function `has_permission()` (007_permissions_system.sql:192-222) evaluates them with deny-first: an effect = 'deny' with equal priority always wins over an allow .

Row Level Security (RLS) is enabled on roles , `user_roles` and `resource_policies` . Even if a SQL query passes with standard user identity, PostgreSQL filters at engine level ? double-curtain behind the application.

Front side: `AbacEngine.check()` does a cache-first with TTL 5 minutes (`engine.ts:30-40`) ? not need query at each render. When the rules change, `invalidate(resourceId)` purges the cache. The front only hides or shows elements ? the final access decision is always on the side server.

Input validation and injection protection

The principle: we never trust the data that comes in. Even if it's our own frontend who sends them.

Zod schemas ? wherever possible, we use zod to validate payloads. Example on the account routes (`account.routes.ts:33-55`):

And the helper who applies them uniformly, in `helpers.ts:70-81`:

-- Role 'user': complete CRUD, but only on its own resources

INSERT INTO public.resource_policies

(role_id, resource_type, resource_name, actions, conditions, effect, priority)

SELECT r.id, '*', '*', ARRAY['select','insert','update','delete'],

'{"owner_only": true}':jsonb,

-- attribute: owner only

'allow', 0

FROM public.roles r WHERE r.name = 'user';

-- Role 'admin': same CRUD, without ownership restrictions

INSERT INTO public.resource_policies (...)

SELECT r.id, '*', '*', ARRAY['select','insert','update','delete'],

'{}':jsonb,

-- no conditions = universal access

'allow', 100

-- priority 100 > 0 ? wins over user

FROM public.roles r WHERE r.name = 'admin';

const passwordSchema = z.string().min(8);

const emailCreateSchema = z.object({

email: z.string().regex(/^[^\s@]+@[^\s@]+\.[^\s@]+\$/),

});

const twoFactorVerifySchema = z.object({

token: z.string().regex(/^[0-9]{6}\$/),

});

Osionos ? Project file

89 / 101

```
hema = z.object({  
  email: z.string().regex(/^[^\s@]+@[^\s@]+\.[^\s@]+$/),  
});  
const twoFactorVerifySchema = z.object({  
  token: z.string().regex(/^[0-9]{6}$/),  
});
```

Osionos ? Project file

89 / 101

```
hema = z.object({  
  email: z.string().regex(/^[\s@]+@[\s@]+\.[\s@]+$/),  
});  
const twoFactorVerifySchema = z.object({  
  token: z.string().regex(/^\d{6}$/),  
});
```

Osionos ? Project file

89 / 101

If a field is missing or incorrectly typed, we return a 400 VALIDATION_FAILED with the details ? the query never reaches the business layer.

NestJS side ? same but with class-validator. We have a global validation pipeline with a config strict (validation.pipe.ts:20-37):

whitelist: true ? any property not declared in the DTO is removed

forbidNonWhitelisted: true ? even worse, it returns a 400 if there are too many fields

transform: true ? type self-coercion (a "42" becomes a 42 if the DTO requests it)

Which means that we cannot inject an isAdmin: true field and hope that it passes through above the DTO. It is cleaned before even reaching the controller.

SQL/NoSQL injection protection

We mainly use MongoDB (via Mongoose and the native driver), so no SQL string concat to fear. But NoSQL injection also exists. The query-router filters dangerous operators before to execute (mongodb.engine.ts:35-72):

And in the collections layer, we explicitly strip the sensitive fields before inserting (collections.service.ts:24-63):

For PostgreSQL queries (GoTrue and permissions side), everything goes through parameterized queries ? This is the default pattern for pg and PostgREST. No concatenation of strings.

On rate limiting: Kong applies it on critical public roads (kong.yml:118-123) ? /auth/v1

is limited to 300 req/min per IP, /rest/v1 to 180/min. This is not fine application throttling, but it covers basic brute force.

Front-end and API protections

CORS ? origin control

CORS is configured at the Kong level, not in each microservice (another advantage of the gateway centralized). Config in kong.track-binocle.yml:24-35:

```
export function parseBody<TSchema extends ZodType>(  
  schema: TSchema, body: unknown, reply: FastifyReply,  
): z.infer<TSchema> | undefined {
```

```
  const parsed = schema.safeParse(body ?? {});
```

```
  if (!parsed.success) {
```

```
    sendError(reply, 400, 'VALIDATION_FAILED', 'Invalid request body', parsed.error.issues);
```

```
    return undefined;
```

```
  }
```

```
  return parsed.data;
```

```
}
```

```
?
```

```
?
```

```
?
```

```
if (!/^[w-]{1,64}$/.test(collectionName)) throw new Error('Invalid collection name');
```

```
delete filter["$where"]; // $where allows you to evaluate JS ? deleted
```

```
const { _id: __, owner_id: __, ...clean } = data;
```

```
// we never let the client write _id or owner_id directly
```

Osionos ? Project file

90 / 101

```
t { _id: __, owner_id: __, ...clean } = data;  
// we never let the client write _id or owner_id directly  
Osionos ? Project file  
90 / 101
```

```
t { _id: __, owner_id: __, ...clean } = data;  
// we never let the client write _id or owner_id directly  
Osionos ? Project file  
90 / 101
```


Origins are placeholders templated at startup from environment variables ?

so in dev we have `https://localhost:5173`, in prod this would be the real domain. No * in production.

How we defend sensitive roads

Back-end side: each protected controller pastes a `@UseGuards(AuthGuard)` (and `RolesGuard` if role required). The `AuthGuard` (`auth.guard.ts`) reads `X-User-Id` injected by Kong and hydrates `req.user`. If the header is absent ? 401. If Kong did not validate the JWT, he would not have added this header ? it is a string of controlled trust.

Front-end side: we use the Zustand store (`useUserStore`) which hydrates from the server to the mount of the App (`App.tsx:1-68`). Protected routes check status before rendering content, otherwise redirect to the login.

Where we store tokens ? complete honesty

Regarding the storage of tokens, it's not perfect:

The refresh token is in cookie `HttpOnly + Secure + SameSite=Lax` (`auth-gateway.mjs:141-147`). That, that's good: an XSS script can't read it, and it's only sent to the original domain.

The access token is manipulated on the client side to sign API requests in `Authorization: Bearer <jwt>` (`client.ts:68`). In practice, we keep this in mind in the Zustand blind. What is stored in `localStorage`, these are context metadata (workspaces, active accounts) ? not the JWT itself. same: see `useUserStore.ts:39-41`.

The compromise: a short access token (1 hour) limits the risk window, and the refresh token in `HttpOnly` blocks XSS theft of the part that really matters (the ability to renew the session). It's a classic trade-off in the SPA ecosystem ? there are stricter architectures (BFF with cookie of session), but it is reasonable for the scope of the project.

Protections against XSS and CSRF

XSS ? Cross-Site Scripting

React, by default, automatically escapes everything rendered in JSX (`{userInput}`). This is the first line of defense, and it's free.

But we have a rich block editor that renders Markdown ? so we generate HTML from input user. There, React can no longer do the job alone. We wrote our own markengine which does the escaping itself (`renderCore.ts:92-103`):

- name: corns

config:

origins:

- `__KONG_CORS_ORIGIN_APP__`

- `__KONG_CORS_ORIGIN_PLAYGROUND__`

- `__KONG_CORS_ORIGIN_STUDIO__`

methods: `[GET, POST, PUT, PATCH, DELETE, OPTIONS]`

credentials: `true`

max_age: `3600`

?

?

Osionos ? Project file

91 / 101

LAYGROUND__

- __KONG_CORS_ORIGIN_STUDIO__

methods: [GET, POST, PUT, PATCH, DELETE, OPTIONS]

credentials: true

max_age: 3600

?

?

Osionos ? Project file

91 / 101

LAYGROUND__

- __KONG_CORS_ORIGIN_STUDIO__

methods: [GET, POST, PUT, PATCH, DELETE, OPTIONS]

credentials: true

max_age: 3600

?

?

Osionos ? Project file

91 / 101

And ? perhaps more importantly ? we have a `sanitizeUrl()` that rejects dangerous patterns (renderCore.ts:109-123):

We even have a test which verifies that `[bad](javascript:alert(1))` transforms into `href="#"` without never let javascript pass: (markengine.test.js:85-90). It protects us against XSS payload the best known Markdown editor.

Honest about the shortcomings: we have not defined a Content-Security-Policy header on the Kong/BaaS side. The

Astro site has a CSP declared in its layout, but the BaaS gateway does not yet impose it globally. This is in-depth protection to be added after auditing external origins (CDN of fonts, endpoints API, assets) to write a CSP that does not break production.

CSRF ? Cross-Site Request Forgery

This is the part where the archi protects a little ?naturally?:

All sensitive API requests use Authorization: Bearer <jwt> ? a custom header that is never sent automatically by the browser. So a cross-origin CSRF request cannot not include the token. This neutralizes the classic CSRF vector.

The only cookie we use (the refresh token) is SameSite=Lax, which means that it is not sent on cross-origin POST requests (and only on top-level GET navigations).

Strict CORS (whitelisted origins) adds an extra layer: even if someone tried, pre-flight CORS would block.

We have not implemented explicit CSRF tokens (synchronize-token / double-submit-cookie style) because that Bearer + SameSite auth already covers the need. This is the standard compromise of modern SPAs.

GDPR compliance

Many projects say ?we are GDPR-compliant? without being able to demonstrate it. This is what is really implemented.

```
const HTML_ESCAPE_PATTERN = /[&<>"]/g;
const HTML_ESCAPE_MAP: Record<string, string> = {
  "&": "&amp;", "<": "&lt;", ">": "&gt;", "\"": "&quot;", "':": "&#39;",
};
export function escapeHtml(value: string): string {
  return value.replaceAll(HTML_ESCAPE_PATTERN, (char) => HTML_ESCAPE_MAP[char]);
}
export function sanitizeUrl(value: string): string {
  const normalized = stripUrlControlAndSpaceChars(trimmed);
  const schemeMatch = /^[a-z][a-z\d+.-]*$/i.exec(normalized);
  if (!schemeMatch) return trimmed;
  const scheme = schemeMatch[1].toLowerCase();
  if (scheme === "http" || scheme === "https" || scheme === "mailto" || scheme ===
  "tel") {
    return trimmed;
  }
  return ""; // everything else (javascript:, data:, etc.) is whitened
}
?
?
?
```

Osionos ? Project file

92 / 101

anything else (javascript:, data:, etc.) is whitened

}

?

?

?

Osionos ? Project file

92 / 101

anything else (javascript:, data:, etc.) is whitened

}

?

?

?

Osionos ? Project file

92 / 101

A dedicated GDPR service in BaaS: `apps/baas/mini-baas-infra/src/apps/gdpr-service/`. He exposes three families of endpoints which correspond to the main GDPR rights.

Right to portability (Article 20) ? complete export of data:

GET `/export` in `export.controller.ts:26-30` returns a structured dump (JSON) of all data associated with the user.

Right to erasure (Article 17, ?right to be forgotten?) ? account deletion with grace period 30 days:

POST `/account/request-deletion` (`account.routes.ts`) mark `pendingDeletionAt = now + 30` days. User can cancel for 30 days via DELETE `/account/request-deletion`. Past this time, a job actually purges the data.

This is important: immediate deletion is good for compliance, but it generates regrets and support tickets. 30 days is a standard at Google/GitHub too.

Consent management (Articles 6-7) ? granular opt-in for non-essential processing:

`/consents` endpoints in `consent.controller.ts` allow the user to accept/refuse separately:

Analytics cookies (deactivated by default)

Personalization cookies (deactivated by default)

Essential cookies (always active, justified by technical necessity)

The corresponding UI is in `CookieSettingsModal` of `SettingsCenter.tsx`.

Limitation of processing & privacy settings:

Toggle ?discoverable profile? ? a user can be invisible in search

Toggle ?view history? ? deactivation of reading tracking

Honest about what's missing:

We do not have an intrusive cookie banner on first load. Preferences change in Settings. For real production, you would probably need a consent banner explicit at the first visit (depending on the jurisdiction).

We have not formalized a textual ?Privacy Policy? or ?Cookie Notice? ? we have the mechanisms, not still the legal documents that accompany them.

Summary of the chapter: we have solid auth (GoTrue + bcrypt + short JWT + refresh HttpOnly), a two-level authorization (RBAC via guards + ABAC via SQL with deny-first), strict side validation API (Zod + class-validator with whitelist), active XSS protection in the Markdown engine, and real GDPR mechanisms (export, deletion at 30 days, granular consent). What remains to be done: CSP global BaaS/Kong side and cookie banner on first load.

CHAPTER 6. Technology monitoring and security

Web security is not a fixed state ? it's a flow. New vulnerabilities come out every week.

Some affect libraries that we use directly. Others give patterns that we would reproduce without knowing it if we did not read them. This chapter documents how we kept ourselves informed

and what we actually learned from it for the project.

?

?

?

?

?

?

?

?

?

?

?

?

Osionos ? Project file

GitHub Actions ? stealing secrets via pull_request_target

The ?pwn request? pattern: an external PR triggers a pull_request_target workflow which has access to the secrets of the repo. The attacker exfiltrates via network calls in the logs. Documented by GitHub Security Lab (Preventing pwn requests) with real cases. It is this type of incident which reinforced the choice to keep application secrets outside of GitHub Actions when possible: the colleague workflow authenticates to Vault via OIDC, writes a temporary .vault/track-binocle-reader.env file, then make all retrieves the necessary .envs without storing a static Vault token in GitHub secrets.

Claude Code ? leak via npm .map file (2026)

In March 2026, Anthropic accidentally released a 59.8 MB source map file (.js.map) in the package @anthropic-ai/claude-code v2.1.88. The file, intended for internal debugging, exposed ~512,000 lines of TypeScript. Cause: The Bun build tool generates source maps by default, and .map was not in .npmignore. (InfoQ, Layer5 blog)

This was not a token leak ? no sensitive user data was exposed. But it illustrates a classic vector: a build artifact that should not be public ends up in a package npm. On this project, vite.config.ts does not explicitly enable production source maps (build.sourcemap is absent, and Vite keeps them disabled by default in prod build).

JWT algorithm confusion

The family of attacks where we change the algorithm of a JWT from RS256 to HS256 and we sign with the key public as HMAC key. Documented in detail by PortSwigger (algorithm-confusion). We are not exposed since we use HS256 with a symmetrical secret only, but understanding this vector has confirmed that the choice of algorithm should not be left to the client side. In the Kong config, the algorithm is forced to HS256 (kong.yml:21) ? no negotiation.

ReDoS via regex in validators

Zod and other validation libraries have had issues with regular expressions catastrophic events on malformed inputs (OWASP ReDoS). We have regex in Zod schemas (account.routes.ts:45) ? nothing complex, but it?s a pattern to watch out for.

Potential flaws and corrections to be made

What was identified on the project as security debt, in order of priority:

Review

Rate limiting Kong ? configured on critical public roads via the rate-limiting plugin (kong.yml:118-123): 300 req/min on /auth/v1 , 180 on /rest/v1 , 120 on WebSocket realtime. This is rate limiting per IP, which covers brute force. What is not covered: attacks distributed multi-IP (no rate limiting per user account, no progressive blocking type CAPTCHA after N failures).

Important

Content-Security-Policy absent on the Kong/BaaS side ? the Astro site defines a CSP, but the BaaS gateway does not yet set a global CSP header. An XSS which would pass on an application surface not covered could therefore benefit from insufficient defense in depth. Correction: define a CSP strict via Kong (response-transformer plugin) after auditing the origins of scripts/fonts.

?

?

Osionos ? Project file

ction: define a CSP

strict via Kong (response-transformer plugin) after auditing the origins of scripts/fonts.

?

?

Osionos ? Project file

95 / 101

ction: define a CSP

strict via Kong (response-transformer plugin) after auditing the origins of scripts/fonts.

?

?

Osionos ? Project file

95 / 101

No explicit cookie banner ? consent preferences exist in the Settings but it
There is no opt-in mechanism on first load. Required in some GDPR jurisdictions.

To watch

npm dependencies ? CI checks frozen installs (`npm ci --ignore-scripts` , `pnpm install --frozen-lockfile`) and the repo contains Dependabot + Renovate, but it is still missing an SCA gate blocking type `npm audit --audit-level=high` or equivalent. On a project with this density of packages, it is a passive risk.

MFA not implemented ? TOTP endpoint returns 501 Not Implemented (`auth-gateway.mjs:1076`).

For admin accounts, the absence of a second factor is an exposure.

Long-lived OAuth tokens ? Google Calendar/Gmail tokens have a long lifespan and are stored on the server side. A storage compromise would expose them.

Conclusion

Doing security monitoring means above all accepting that we code in an environment that changes quickly. The tools we use (npm, GoTrue, Kong, GitHub Actions) have all had documented incidents. Read these incidents regularly helps to anticipate rather than react.

On this project, the day before had a concrete impact: the choice of Vault for secrets, the forcing of the JWT algorithm on the Kong side, and the removal of \$where in the MongoDB query-router are all decisions that come from patterns read in real vulnerability reports ? not just good ones generic practices.

CHAPTER 7. Conclusion

Before closing this file, I want to take a moment to thank the people who counted in this project, and more broadly in this year at 42. What we have built here is not just code.

It's hours of debugging at 2 a.m., arch choices that we turned around in all directions, moments when we really didn't know if it was the right direction. And yet we have moved forward.

What this project taught me personally, beyond technology:

the design of distributed architectures: assembling specialized services (Kong, GoTrue, PostgREST, NestJS, realtime-agnostic, Trino) so that it holds together, that it is coherent, secure and maintainable ? this is a way of thinking that I didn't have at all before this project;
leadership: managing four people, coordinating roles, arbitrating priorities when everyone world is not available at the same time ? it's much more complicated than writing code, and it's undoubtedly what trained me the most;

software quality: I learned a lot about data modeling, web security, and this that it really means to build something that lasts over time and not just something thing that turns.

A special thank you to Vadim, who never gave up. This project is hard. There are weeks when we don't see not where we're going. Vadim was there with a consistency and rigor that really counted, and I am sincerely proud of what we have built together.

Last thing, and I want to be honest about this: on the day of the exam, the project may still be in progress. construction site. We don't know if we will have released the MVP that we initially imagined. But what I know is that this path was worth it. It doesn't matter what the demo shows that day.

?

?

?

?

?

?

?

Osionos ? Project file

ws that day.

?

?

?

?

?

?

?

Osionos ? Project file

96 / 101

ws that day.

?

?

?

?

?

?

?

Osionos ? Project file

96 / 101

Category

Resource

What we learn there

Videos / Channels

Fireship

Tech explained in 100 seconds

Videos / Channels

t3.gg ? Theo

React, TypeScript, frontend architecture

Videos / Channels

WebDevSimplified

Web concepts explained simply

Videos / Channels

Kevin Powell ? CSS

Master CSS in depth

Videos / Channels

David J. Malan ? Vibe Coding

Interview

Harvard professor on the use of AI in
learning

Osionos ? Project file

100 / 101

Category

Resource

What we learn there

Videos / Channels

Fireship

Tech explained in 100 seconds

Videos / Channels

t3.gg ? Theo

React, TypeScript, frontend architecture

Videos / Channels

WebDevSimplified

Web concepts explained simply

Videos / Channels

Kevin Powell ? CSS

Master CSS in depth

Videos / Channels

David J. Malan ? Vibe Coding

Interview

Harvard professor on the use of AI in
learning

Osionos ? Project file

100 / 101